

UNIVERSITATEA BABEȘ-BOLYAI
FACULTATEA DE BUSINESS

Lucrare de disertație

Coordonator științific,

Prof.univ.dr. *Ioan-Cristian Chifu*

Lect.univ.dr. *Liliana Manciu-Guran*

Digitally signed by

IOAN-CRISTIAN CHIFU

Date: 2026.06.11 11:17:20 +03'00'

Digitally signed by

LILIANA MANCIU GURAN

Date: 2026.06.14 09:52:01 +03'00'

Student,

Cojocaru Eugeniu

Cluj-Napoca

2026

UNIVERSITATEA BABEȘ-BOLYAI
FACULTATEA DE BUSINESS
SPECIALIZAREA
ADMINISTRAREA AFACERILOR

Lucrare de disertație
Automatizarea Analizei Datelor prin
Integrarea Tehnologiilor Web Moderne
Office.js

Coordonator științific,

Prof.univ.dr. *Ioan-Cristian Chifu*

Lect.univ.dr. *Liliana Manciu-Guran*

Student,

Cojocaru Eugeniu

Cluj-Napoca

2026

ABSTRACT

Prezenta lucrare de disertație abordează problema automatizării și eficientizării analizei econometrice în mediul de afaceri și academic, propunând o tranziție de la metodele statistice tradiționale (bazate pe calcul manual laborios sau pe macrocomenzi VBA vulnerabile) către tehnologiile web moderne. Obiectivul central al cercetării constă în proiectarea, implementarea și validarea empirică a unui instrument software, de tip Office Add-in, integrat nativ în Microsoft Excel. Acest instrument este capabil să execute modele de regresie liniară multiplă (*OLS*) și să genereze rapoarte interpretative automate, direct în limbaj natural.

Din perspectivă teoretică, lucrarea fundamentează abordarea matriceală a *Metodei Celor Mai Mici Pătrate*, analizând riguros ipotezele clasice *Gauss-Markov*. O atenție deosebită este acordată econometriei variabilelor calitative, detaliind procesul de conversie a acestora în regresori numerici și aplicarea strictă a regulii celor $m-1$ variabile pentru a preveni coliniaritatea perfectă (*Capcana Variabilelor Dummy*).

Pentru a transpune această rigoare matematică în cod, arhitectura aplicației a fost dezvoltată modular, utilizând ecosistemul React, Fluent UI și API-ul modern Office.js. Cercetarea rezolvă o problemă fundamentală a calculului computațional în mediul browserelor web: limitările de precizie ale standardului IEEE 754 (reprezentarea în virgulă mobilă). Prin integrarea bibliotecilor de calcul științific și utilizarea structurilor de date *BigNumber* la o precizie de 64 de zecimale, motorul matematic propriu garantează lipsa erorilor de propagare. Mai mult, algoritmul eludează inversarea explicită a matricelor informaționale, un proces recunoscut pentru instabilitatea sa. În schimb, estimarea coeficienților și a erorilor standard se realizează exclusiv printr-o metodă avansată de analiză numerică: *descompunerea LU*, care conferă sistemului robustețe chiar și în fața datelor defectuoase (matrice singulare).

Inovația principală a instrumentului dezvoltat o reprezintă modulul de interpretare dinamică. Pe lângă generarea tabelară a metricilor standard (*coeficienți*, *p-values*, R^2), aplicația scanează metadatele și traduce rezultatele numerice în propoziții economice explicative. Acestea sunt corelate vizual, printr-un sistem cromatic determinist, cu pragurile de semnificație statistică, adaptându-se dinamic la tipul modelului (Liniar, Log-Liniar etc.).

Auditarea și validarea algoritmului prin analiză comparativă cu modulul oficial Microsoft (*Data Analysis Toolpak*) au demonstrat o acuratețe perfectă a zecimalelor și o superioritate clară în tratarea situațiilor limită și diagnosticarea *multicoliniarității* (prin indici VIF). În concluzie, lucrarea demonstrează că utilizarea tehnologiilor asincrone transformă foaia de calcul dintr-un simplu spațiu de stocare într-o platformă educațională și de cercetare robustă, democratizând accesul la statistica avansată printr-o soluție ce necesită doar câteva interacțiuni din partea utilizatorului.

CUPRINS

CAPITOLUL 1. INTRODUCERE	1
1.1. CONTEXT ȘI SCURT REZUMAT AL CERCETĂRII	1
1.2. SCOPUL CERCETĂRII	1
1.3. METODOLOGIA CERCETĂRII	2
1.4. CONCLUZIILE STUDIULUI ȘI PROPUNERI/RECOMANDĂRI	2
CAPITOLUL 2. STUDIU BIBLIOGRAFIC ȘI FUNDAMENTE TEORETICE	4
2.1. MODELUL DE REGRESIE LINIARĂ MULTIPLĂ ÎN ABORDAREA MATRICEALĂ	4
2.1.1. FORMULAREA MATEMATICĂ A METODEI OLS SUB FORMĂ DE MATRICE	4
2.1.2. IPOTEZELE FUNDAMENTALE GAUSS-MARKOV ȘI DEDUCEREA TEORETICĂ A ESTIMATORILOR (β)	6
2.1.3. EVALUAREA CALITĂȚII MODELELOR: COEFICIENTUL R^2 , R^2 AJUSTAT ȘI ERORILE STANDARD	9
2.2. ECONOMETRIA VARIABILELOR CALITATIVE (DUMMY)	11
2.2.1. CONCEPTUL DE VARIABILĂ CALITATIVĂ ȘI MODELELE ANOVA/ANCOVA ÎN LITERATURA DE SPECIALITATE	11
2.2.2. REGULA CELOR $M-1$ VARIABILE ȘI ANALIZA TEORETICĂ A COLINIARITĂȚII PERFECTE (CAPCANA VARIABILELOR DUMMY)	13
2.3. ANALIZĂ NUMERICĂ COMPUTAȚIONALĂ ȘI DESCOMPUNEREA LU	15
2.4. TEHNOLOGII MODERNE PENTRU AUTOMATIZAREA MEDIILOR DE CALCUL	16
2.4.1. EVOLUȚIA SOLUȚIILOR DE AUTOMATIZARE ÎN FOILE DE CALCUL: DE LA LIMBAJUL VBA LA ECOSISTEMUL WEB MODERN OFFICE.JS	16
2.4.2. MANAGEMENTUL PRECIZIEI DATELOR ÎN MEDII RUNTIME ASINCRONE (REPREZENTAREA NUMERELOR ÎN JAVASCRIPT)	18
CAPITOLUL 3. PROIECTARE DE DETALIU ȘI IMPLEMENTAREA SISTEMULUI SOFTWARE	20
3.1. ARHITECTURA DETALIATĂ ȘI FLUXUL DE DATE AL APLICAȚIEI	20
3.1.1. STRUCTURA MODULARĂ A ADD-IN-ULUI	20
3.1.2. FLUXUL DE DATE	22
3.2. IMPLEMENTAREA ALGORITMILOR ÎN JAVASCRIPT	23
3.2.1. DEZVOLTAREA MODULULUI DE PRE-PROCESARE A DATELOR	23
3.2.2. PROGRAMAREA MOTORULUI ECONOMETRIC OLS	25

3.2.3. LOGICA DE TRANSFORMARE DINAMICĂ A DATELOR ÎN FUNCȚIE DE TIPUL MODELULUI (LINIAR, LOG-LINIAR)	27
3.3. PROIECTAREA UI ȘI A MODULULUI DE INTERPRETARE AUTOMATĂ	28
3.3.1. DESIGNUL CONTEXTUAL AL PANOULUI DE LUCRU (TASKPANE HTML/CSS) ȘI GESTIUNEA STĂRILOR ÎN UI	28
3.3.2. ALGORITMUL DE GENERARE DINAMICĂ A TEXTELOR DE INTERPRETARE ECONOMETRICĂ ÎN FUNCȚIE DE PRAGURILE DE SEMNIFICAȚIE (P-VALUES)	31
3.4. TESTARE, VALIDARE ȘI ANALIZĂ COMPARATIVĂ (VALIDARE EMPIRICĂ)	33
3.4.1. STRATEGIA DE TESTARE A ALGORITMULUI PROPRIU PE SETURI DE DATE CU NUMĂR VARIABIL DE REGRESORI	33
3.4.2. AUDITAREA REZULTATELOR: ANALIZĂ COMPARATIVĂ ȘI CONFRUNTAREA ZECIMALELOR CU MODULUL OFICIAL MICROSOFT (DATA ANALYSIS TOOLPAK)	34
3.4.3. GESTIONAREA SITUAȚIILOR LIMITĂ ÎN EXECUȚIE (EDGE-CASES) ȘI TRATAREA ERORILOR	36
<u>CAPITOLUL 4. MANUAL DE UTILIZARE AL APLICAȚIEI</u>	<u>38</u>
4.1. CERINȚE TEHNICE ȘI COMPATIBILITATE	38
4.2. GHID DE UTILIZARE PAS CU PAS	39
4.2.1. PREGĂTIREA ȘI STRUCTURAREA DATELOR	39
4.2.2. CONFIGURAREA ȘI LANSAREA ANALIZEI	40
4.2.3. CITIREA ȘI INTERPRETAREA RAPOARTELOR GENERATE	41
<u>CAPITOLUL 5. CONCLUZII, LIMITĂRI ȘI DIRECȚII VIITOARE</u>	<u>43</u>
5.1. CONCLUZII PRIVIND OBIECTIVELE REALIZATE	43
5.2. LIMITĂRI TEHNICE ȘI ECONOMETRICE IDENTIFICATE	43
5.3. PROPUNERI ȘI DIRECȚII VIITOARE DE DEZVOLTARE	44
<u>CAPITOLUL 6. BIBLIOGRAFIE</u>	<u>46</u>

CAPITOLUL 1. INTRODUCERE

1.1. Context și scurt rezumat al cercetării

Evoluția rapidă a tehnologiei informației a schimbat semnificativ modul în care sunt prelucrate și analizate datele, atât în mediul academic, cât și în cel economic. Analiza econometrică și modelarea statistică rămân instrumente esențiale pentru înțelegerea fenomenelor economice și pentru fundamentarea deciziilor, însă partea practică a acestor procese implică deseori activități repetitive, scriere manuală de cod și interacțiunea cu interfețe rigide. Toate acestea consumă timp și cresc riscul apariției erorilor umane.

Pornind de la această realitate, lucrarea de față urmărește să aducă împreună modelarea econometrică tradițională și dezvoltarea de aplicații software moderne, cu scopul de a simplifica și automatiza etapele de lucru. Practic, am transpus conceptele teoretice într-un mediu web accesibil și ușor de utilizat, finalizând proiectul prin implementarea unui instrument de analiză integrat cu platforma Microsoft Excel, menit să reducă efortul utilizatorului și să eficientizeze întregul proces.

1.2. Scopul cercetării

Scopul principal al acestei cercetări este automatizarea analizei datelor printr-un flux de lucru fluid, inspirat de instrumentele consacrate, dar îmbunătățit din perspectiva experienței utilizatorului. Am urmărit proiectarea unui sistem care să genereze rezultate econometrice avansate - regresii liniare și log-liniare, tratarea variabilelor dummy etc. - în doar câteva click-uri. În practică, asta înseamnă eliminarea necesității de a scrie manual ecuații (boilerplate) pentru fiecare set de date analizat, reducând astfel timpul de lucru și riscul erorilor umane.

Un obiectiv secundar, cu valoare inovatoare, este integrarea unui modul dinamic care produce interpretări statistice la comandă. Prin această componentă, aplicația nu rămâne doar un calculator numeric, ci devine un asistent educațional: explică pașii metodologici, verifică validitatea modelelor și interpretează semnificația economică a rezultatelor (de exemplu, interpretarea coeficienților, testele de semnificație *p-value*, erorile standard). Această funcționalitate este gândită pentru a sprijini studenții și cercetătorii aflați la început de drum, oferindu-le explicații clare și aplicate direct în fluxul de lucru.

1.3. Metodologia cercetării

Metodologia urmărită combină cercetarea teoretică cu ingineria software aplicată, păstrând în introducere o descriere succintă, iar detaliile tehnice urmând a fi dezvoltate în capitolele dedicate. În prima fază s-a realizat un studiu riguros al metodelor de estimare econometrică, cu un accent deosebit pe Metoda Celor Mai Mici Pătrate (OLS) tratată prin prisma algebrei matriceale. Am preferat formalismul matriceal în locul notațiilor clasice bazate pe sume scalare pentru că exprimarea compactă $Y = X\beta + u$ și formula estimatorului $\beta = (X^T X)^{-1} X^T Y$ oferă o legătură directă între teoria matematică și structurile de date folosite în programare; astfel ecuațiile se pot mapa nativ pe tablouri bidimensionale (vectori de vectori) și pe operații vectorizate, ceea ce simplifică implementarea și crește eficiența calculului.

Această abordare a facilitat utilizarea librăriilor numerice de înaltă precizie (de exemplu *mathjs*) și adoptarea unor tehnici numerice stabile (evitarea inversării explicite a matricelor, folosirea descompunerilor numerice) pentru a păstra corectitudinea și robustețea rezultatelor în mediul *JavaScript*. Partea practică a modelării algoritmice a fost completată de o analiză a ecosistemului tehnologic, cu accent pe arhitectura API-ului *Office.js*: am verificat modul în care datele structurate matriceal în memoria aplicației pot fi injectate asincron și eficient în celulele Excel, astfel încât fluxul de lucru să rămână familiar utilizatorului, dar să beneficieze de puterea calculului automatizat. În introducere rămâne această sinteză; explicațiile matematice și detaliile de implementare vor fi dezvoltate pas cu pas în capitolele următoare.

1.4. Concluziile studiului și propuneri/recomandări

În urma demersului, concluzia principală este că obiectivele propuse au fost atinse: am dezvoltat un add-in pentru Excel complet funcțional, integrat în arhitectura aplicației, care transformă seturi brute de date în rapoarte formate și interpretări statistice cu intervenție minimă din partea utilizatorului. Implementarea dovedește atât viabilitatea tehnică a soluției (algoritmi numerici stabili rulați în mediul web/Excel), cât și utilitatea practică: reducerea timpului de lucru, scăderea erorilor umane și facilitarea trecerii de la teorie la aplicare.

Recomandarea principală este introducerea instrumentului în activitatea didactică (curs/seminar de econometrie) pentru a sprijini procesul de învățare aplicată: studenții pot

vedea imediat legătura dintre date, model și interpretare, fără a fi blocați de detalii repetitive. Ca direcții de dezvoltare viitoare propun extinderea modulului de diagnosticare (teste pentru autocorelare, multicolaritate), îmbunătățirea interfeței pentru gestionarea variabilelor și integrarea unor opțiuni avansate de export și raportare, astfel încât instrumentul să devină o soluție completă pentru analiza avansată a datelor în mediul universitar și de cercetare.

CAPITOLUL 2. STUDIU BIBLIOGRAFIC ȘI FUNDAMENTE TEORETICE

2.1. Modelul de regresie liniară multiplă în abordarea matriceală

2.1.1. Formularea matematică a metodei OLS sub formă de matrice

Modelul clasic de regresie liniară multiplă descrie relația structurală dintre o variabilă dependentă (explicată) și un ansamblu de variabile independente (explicative sau regresori). În abordarea tradițională, scalară, ecuația econometrică pentru fiecare observație individuală i din eșantion se definește sub forma:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_k x_{ik} + u_i, \quad i = 1, 2, \dots, n$$

unde n reprezintă numărul total de observații din eșantion, iar k este numărul de variabile independente incluse în model (fără a număra termenul liber). Deși această notație scalară este utilă pentru înțelegerea intuitivă a impactului fiecărui regresor, extinderea analizei la modele complexe, cu zeci de variabile și mii de observații, devine extrem de greoaie. Pentru a depăși această barieră computațională și conceptuală, literatura econometrică contemporană utilizează pe scară largă formalismul algebrei liniare.

Pentru a grupa toate cele n ecuații individuale într-o singură structură compactă, se definesc următoarele entități matriceale și vectoriale:

- **Y (vectorul variabilei dependente)** = vector coloană de dimensiune $n \times 1$, care reunește toate valorile observate ale fenomenului analizat:

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

- **X (matricea regresorilor)** = matrice de dimensiune $n \times (k+1)$. Prima coloană a acestei matrice este formată exclusiv din valori de 1, având rolul de a multiplica termenul liber (β_0) în sistemul de ecuații. Următoarele k coloane conțin valorile observate pentru variabilele independente:

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \dots & x_{1k} \\ 1 & x_{21} & x_{22} & \dots & x_{2k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nk} \end{pmatrix}$$

- **β (vectorul parametrilor necunoscuți)** = vector coloană de dimensiune $(k+1) \cdot I$, care stochează coeficienții ce trebuie estimați (interceptul și pantele parțiale):

$$\beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_k \end{pmatrix}$$

- **u (vectorul erorilor reziduale/stohastice)** = vector coloană de dimensiune $n \cdot I$, care înglobează perturbațiile sau factorii neobservați pentru fiecare înregistrare:

$$u = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix}$$

Prin combinarea acestor structuri pe baza regulilor de înmulțire a matricelor, modelul liniar multiplu poate fi exprimat sub forma sa condensată:

$$Y = X\beta + u$$

Principiul fundamental al metodei celor mai mici pătrate (Ordinary Least Squares - OLS) constă în determinarea acelui vector de estimatori, notat cu $\hat{\beta}$, care minimizează suma pătratelor abaterilor dintre valorile reale înregistrate (Y) și valorile estimate sau prezise de model ($\hat{Y} = X\hat{\beta}$).

În termeni algebrici, funcția obiectiv de minimizat (S) reprezintă suma pătratelor reziduurilor. Aceasta se poate scrie compact ca produsul scalar dintre vectorul transpus al erorilor și vectorul original al erorilor:

$$S(\beta) = \sum_{i=1}^n u_i^2 = u^T u$$

Înlocuind vectorul erorilor din ecuația de definiție ($u = Y - X\beta$), funcția de minimizat capătă următoarea formă:

$$S(\beta) = (Y - X\beta)^T (Y - X\beta)$$

Aplicând proprietatea de distributivitate a operației de transpunere a matricelor, expresia se dezvoltă astfel:

$$S(\beta) = (Y^T - \beta^T X^T)(Y - X\beta) = Y^T Y - \beta^T X^T Y - Y^T X\beta + \beta^T X^T X\beta$$

Deoarece produsul $\beta^T X^T Y$ reprezintă o matrice cu dimensiunea 1×1 (adică un scalar), valoarea sa este identică cu cea a transpusului său: $(\beta^T X^T Y)^T = Y^T X\beta$. Acest lucru

permite restrângerea celor doi termeni din mijloc, ecuația finală a sumei pătratelor reziduurilor devenind:

$$S(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X \beta$$

Pentru a determina vectorul de coeficienți $\hat{\beta}$ care minimizează această funcție, se aplică condiția de ordinul întâi (calcularea gradientului în raport cu vectorul β și anularea acestuia):

$$\frac{\partial S}{\partial \beta} = -2X^T Y + 2X^T X \beta = 0$$

Prin împărțirea ecuației cu constanta 2 și rearanjarea termenilor, se obține sistemul fundamental al ecuațiilor normale:

$$X^T X \beta = X^T Y$$

Pentru ca acest sistem de ecuații liniare să admită o soluție unică, este necesar ca matricea pătratică $X^T X$ de dimensiune $(k+1) \times (k+1)$, să fie nesingulară. Din punct de vedere statistic, acest lucru presupune că matricea X are rang maxim ($\text{rang}(X) = k+1$), ceea ce garantează absența multicolinearității perfecte între variabilele independente.

Dacă această ipoteză este îndeplinită, matricea $X^T X$ este inversabilă. Înmulțind la stânga întreaga expresie cu inversa sa, notată cu $(X^T X)^{-1}$, se izolează vectorul estimatorilor, obținându-se formula clasică OLS în formă matriceală:

$$(X^T X)^{-1} X^T X \beta = (X^T X)^{-1} X^T Y$$

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

Această ecuație elegantă constituie nucleul analizei econometrice moderne. Ea demonstrează cum, prin intermediul algebrei liniare, calculul parametrilor de regresie se reduce la o succesiune ordonată de operații matriceale standard (transpunere, înmulțire și inversare), oferind fundamentul matematic optim pentru implementarea algoritmilor de calcul pe sistemele informatice.

2.1.2. Ipotezele fundamentale Gauss-Markov și deducerea teoretică a estimatorilor (β)

Obținerea matematică a estimatorului OLS, $\hat{\beta} = (X^T X)^{-1} X^T Y$, reprezintă doar fundamentul algebric al regresiei liniare. Pentru ca acest vector de estimatori să dețină proprietăți statistice optime și să poată fi utilizat în inferența econometrică (pentru a trage concluzii valabile despre populația generală pe baza eșantionului), modelul trebuie să

respecte un set strict de condiții teoretice. Aceste condiții sunt cunoscute în literatura de specialitate sub denumirea de **Ipotezele Clasice ale modelului liniar** sau **Ipotezele Gauss-Markov**. În abordarea matriceală, ipotezele fundamentale se formulează astfel:

1. Liniaritatea în parametri: Modelul econometric poate fi exprimat sub forma matriceală $Y = X\beta + u$. Această ipoteză stipulează că variabila dependentă este o funcție liniară de parametrii β și de termenul de eroare u . Variabilele independente X pot suferi transformări neliniare (cum ar fi ridicarea la pătrat sau logaritmarea), însă parametrii estimați trebuie să rămână la puterea întâi.

2. Matricea X este deterministă sau strict exogenă: Valorile variabilelor explicative sunt considerate fixe în eșantioane repetate sau, în modelele mai relaxate, independente de termenul de eroare. Matematic, valoarea așteptată (speranța matematică) a erorilor, condiționată de matricea X , este nulă: $E(u|X) = 0$. Această ipoteză garantează că nu există o informație sistematică rămasă necapturată de regresori care să fie corelată cu variabilele independente.

3. Absența autocorelării (Erori sferice): Varianța termenilor de eroare este constantă pentru toate observațiile (σ^2), iar erorile asociate unor observații distincte sunt necorelate. Formulată matriceal, matricea de varianță-covarianță a erorilor este proporțională cu matricea identitate de ordinul $n(I_n)$:

$$Var(u|X) = E(uu^T|X) = \sigma^2 I_n$$

4. Absența multicolinearității perfecte: Matricea X , de dimensiune $n \times (k+1)$, are rang coloană maxim ($rang(X) = k+1$). Aceasta implică faptul că nicio variabilă independentă nu poate fi exprimată ca o combinație liniară perfectă a celorlalte variabile din model, asigurând astfel că matricea $(X^T X)$ este inversabilă. Dacă aceste ipoteze sunt respectate, se poate deduce teoretic că estimatorul OLS posedă două proprietăți statistice deosebit de importante: este nedeplasat (unbiased) și eficient (are varianță minimă).

Un estimator este considerat nedeplasat dacă, în medie, valoarea sa estimată pe eșantioane repetate egalează valoarea reală a parametrului din populație. Pentru a demonstra acest lucru, se pornește de la formula estimatorului OLS:

$$\hat{\beta} = (X^T X)^{-1} X^T Y.$$

Se înlocuiește vectorul Y cu structura sa teoretică din ipoteza liniarității ($Y = X\beta + u$):

$$\hat{\beta} = (X^T X)^{-1} X^T (X\beta + u)$$

Desfăcând paranteza și aplicând proprietatea de distributivitate:

$$\hat{\beta} = (X^T X)^{-1} X^T X \beta + (X^T X)^{-1} X^T u$$

Deoarece produsul unei matrice cu inversa sa generează matricea identitate, $(X^T X)^{-1} X^T X = 1$, ecuația se simplifică astfel:

$$\hat{\beta} = \beta + (X^T X)^{-1} X^T u$$

Pentru a evalua nedeplasarea, se aplică operatorul speranței matematice (E) condiționat de matricea X :

$$E(\hat{\beta}|X) = E(\beta|X) + E[(X^T X)^{-1} X^T u|X]$$

Deoarece β este o constantă (parametrul real al populației) și X este asumat ca fiind exogen (tratabil ca o constantă în condiționare), operatorul de valoare așteptată se aplică doar termenului de eroare u :

$$E(\hat{\beta}|X) = \beta + (X^T X)^{-1} X^T E(u|X)$$

Conform ipotezei 2 ($E(u|X) = 0$), termenul al doilea se anulează, rezultând fundamentul validității modelului:

$$E(\hat{\beta}|X) = \beta$$

Această deducție demonstrează matematic că metoda OLS oferă estimatori centrați pe valoarea reală, adică nedeplasați.

Pe lângă acuratețea estimării, analiza econometrică este interesată de precizia (dispersia) estimatorilor. Matricea de varianță-covarianță a estimatorilor OLS se calculează evaluând valoarea așteptată a abaterilor la pătrat față de medie:

$$Var(\hat{\beta}|X) = E[(\hat{\beta} - \beta)(\hat{\beta} - \beta)^T | X]$$

Din ecuațiile anterioare se cunoaște că diferența $\hat{\beta} - \beta$ este exact $(X^T X)^{-1} X^T u$. Substituind și aplicând operația de transpunere:

$$Var(\hat{\beta}|X) = E[((X^T X)^{-1} X^T u)(u^T X (X^T X)^{-1}) | X]$$

$$Var(\hat{\beta}|X) = (X^T X)^{-1} X^T E(uu^T | X) X (X^T X)^{-1}$$

Înlocuind matricea de varianță a erorilor cu expresia sa din ipoteza 3 ($E(uu^T | X) = \sigma^2 I_n$), obținem:

$$Var(\hat{\beta}|X) = (X^T X)^{-1} X^T (\sigma^2 I_n) X (X^T X)^{-1}$$

$$Var(\hat{\beta}|X) = \sigma^2 (X^T X)^{-1} X^T X (X^T X)^{-1}$$

Simplificând expresia centrală $(X^T X)^{-1} X^T X = 1$, se deduce formula standard pentru matricea de varianță-covarianță a coeficienților:

$$Var(\hat{\beta}|X) = \sigma^2 (X^T X)^{-1}$$

Această matrice conține pe diagonala principală varianțele fiecărui parametru estimat, din care se extrag erorile standard ale coeficienților prin extragerea radicalului, acestea fiind esențiale în calcularea statisticilor testului t și a intervalelor de încredere.

Teorema Gauss-Markov sintetizează aceste concluzii matematice într-un singur enunț fundamental: Dacă ipotezele 1-4 sunt îndeplinite, atunci estimatorii obținuți prin metoda OLS sunt cei mai buni estimatori liniari nedeplasați (BLUE - Best Linear Unbiased Estimators). Calificativul de "cel mai bun" se referă direct la principiul eficienței: în clasa tuturor estimatorilor posibili care sunt și liniari și nedeplasați, estimatorii OLS posedă varianța minimă, oferind așadar cel mai ridicat grad de precizie.

2.1.3. Evaluarea calității modelelor: Coeficientul R^2 , R^2 ajustat și erorile standard

După determinarea vectorului de estimatori $\hat{\beta}$ prin metoda celor mai mici pătrate (OLS), un pas fundamental în analiza econometrică este evaluarea gradului în care modelul teoretic reușește să explice variația datelor observate în realitate. O estimare matematic corectă nu garantează implicit un model util; prin urmare, validarea performanței acestuia se realizează printr-un set de metrici statistice de bonitate a potrivirii. Cele mai importante instrumente în acest sens sunt coeficientul de determinație, varianta sa ajustată și erorile standard.

Pentru a defini metricile de evaluare, literatura de specialitate pornește de la descompunerea variației totale a variabilei dependente Y . Variația totală a valorilor observate în jurul mediei eșantionului ($\bar{y}$) este cunoscută sub denumirea de **Suma Pătratelor Totale (SST - Total Sum of Squares)** și se compune din două elemente distincte:

1. **Suma Pătratelor Explicate (SSR - Regression Sum of Squares)** = partea din variația totală a lui Y care este capturată (explicată) de modelul de regresie, adică de variabilele independente X .

2. **Suma Pătratelor Reziduurilor (SSE - Error Sum of Squares)** = variația inexplicată, atribuită termenului de eroare u , generată de factori omiși din model sau de caracterul aleatoriu al datelor.

Identitatea fundamentală a analizei de regresie se exprimă astfel: $SST = SSR + SSE$.

Coeficientul de determinație, notat cu R^2 , este principala măsură a calității ajustării liniare. El măsoară proporția din variația totală a variabilei dependente care este explicată

de variația variabilelor independente din model. Din punct de vedere matematic, se calculează ca raportul dintre suma pătratelor explicate și suma pătratelor totale:

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

Valoarea lui R^2 este întotdeauna cuprinsă în intervalul $[0, 1]$. Un R^2 apropiat de 1 indică o putere explicativă ridicată (modelul se pliază perfect pe datele observate), în timp ce o valoare apropiată de 0 sugerează că variabilele independente alese nu explică aproape deloc comportamentul variabilei dependente.

Deși este un indicator extrem de popular, coeficientul R^2 prezintă o limitare majoră în contextul regresiei multiple: valoarea sa nu scade niciodată (și de cele mai multe ori crește) odată cu adăugarea de noi variabile independente în model, chiar dacă acele variabile sunt irelevante din punct de vedere economic. Acest fenomen poate conduce la construirea unor modele artificial supra-dimensionate.

Pentru a corecta deficiența indicatorului clasic, econometria introduce coeficientul de determinație ajustat, notat adesea cu $\bar{R}^2$ sau R^2_{adj} . Acesta aplică o penalizare matematică pentru fiecare variabilă suplimentară adăugată în model care nu aduce un aport explicativ semnificativ. Penalizarea se realizează prin încorporarea gradelor de libertate asociate fiecărei sume de pătrate:

$$\bar{R}^2 = 1 - \frac{SSE/(n - k - 1)}{SST/(n - 1)}$$

Unde n reprezintă numărul de observații, iar k este numărul de variabile independente. Formula poate fi rescrisă în funcție de R^2 clasic sub forma:

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - k - 1}$$

Spre deosebire de R^2 , coeficientul ajustat poate să scadă atunci când se introduce o nouă variabilă independentă dacă valoarea explicativă marginală a acesteia nu compensează pierderea unui grad de libertate. Astfel, $\bar{R}^2$ este un instrument mult mai robust pentru compararea modelelor alternative care prezintă un număr diferit de regresori.

În timp ce coeficienții R^2 evaluează modelul în ansamblul său, erorile standard oferă informații vitale despre precizia la nivel micro (atât a modelului, cât și a fiecărui parametru individual).

1. Eroarea Standard a Estimării (ESE): ESE măsoară împrăștierea valorilor reale în jurul dreptei (sau hiperplanului) de regresie estimate. Ea reprezintă o estimare a deviației standard

a termenului de eroare din populație (σ) și este definită matematic ca rădăcina pătrată a dispersiei reziduale:

$$ESE = \hat{\sigma} = \sqrt{\frac{SSE}{n - k - 1}}$$

Spre deosebire de R^2 (care este un procent fără dimensiune), ESE este exprimată în aceleași unități de măsură ca și variabila dependentă Y . O valoare mai mică a ESE indică faptul că, în medie, predicțiile modelului sunt mai apropiate de valorile reale înregistrate.

2. Erorile Standard ale Coeficienților Estimati: Pentru a realiza inferențe statistice (validarea ipotezelor), este necesar să se cunoască precizia fiecărui coeficient estimat $\hat{\beta}_j$. Aceasta se obține prin extragerea radicalului din elementele aflate pe diagonala principală a matricei de varianță-covarianță a estimatorilor OLS:

$$se(\hat{\beta}_j) = \sqrt{Var(\hat{\beta}_j)} = \sqrt{\hat{\sigma}^2((X^T X)^{-1})_{jj}}$$

Aceste erori standard ale coeficienților, $se(\hat{\beta}_j)$, joacă un rol central în econometrie. Ele constituie numitorul în calculul statisticii testului Student ($tstat = \frac{\hat{\beta}_j}{se(\hat{\beta}_j)}$). O eroare standard redusă relativ la mărimea coeficientului indică o estimare precisă, ceea ce conduce la un nivel de semnificație statistică puternic (un p -value scăzut), permițând respingerea ipotezei nule conform căreia variabila independentă respectivă nu ar avea nicio influență.

2.2. Econometria Variabilelor Calitative (Dummy)

2.2.1. Conceptul de variabilă calitativă și modelele ANOVA/ANCOVA în literatura de specialitate

În analiza econometrică tradițională, atenția este adesea orientată către fenomene și indicatori exprimați numeric, precum veniturile, volumul producției, rata inflației sau costurile operaționale. Cu toate acestea, realitatea economică și socială este profund influențată de o multitudine de factori care nu posedă o scară naturală de măsurare cantitativă. Acești factori, cunoscuți în literatura de specialitate drept variabile calitative, atributive sau categoriale, exprimă prezența sau absența unui anumit atribut ori apartenența unui subiect la o anumită categorie. Exemple clasice includ genul unei persoane, nivelul ultimelor studii absolvite (licență, master, doctorat), localizarea geografică (mediul urban

sau rural), starea tehnică a unui activ (excelentă, bună, acceptabilă) ori contextul macroeconomic general (perioadă de stabilitate sau an de criză economică). Pentru a putea integra aceste atribute non-numerice în ecuațiile riguroase ale metodei melor mai mici pătrate (OLS), teoria econometrică apelează la construcția unor variabile artificiale binare, denumite variabile dummy (sau variabile indicatoare). O variabilă dummy reține doar valorile discrete 0 și 1, unde valoarea 1 semnalează prezența atributului analizat, iar valoarea 0 indică absența acestuia. Din punct de vedere formal, transpunerea unei stări calitative într-un regresor econometric se definește prin funcția indicator:

$$D_i = \begin{cases} 1, & \text{dacă observația } i \text{ posedă atributul} \\ 0, & \text{dacă observația } i \text{ nu posedă atributul} \end{cases}$$

În funcție de modul în care aceste variabile artificiale sunt combinate cu regresorii cantitativi standard în cadrul aceleiași ecuații, literatura de specialitate clasifică modelele econometrice în două mari categorii structurale: modele ANOVA și modele ANCOVA.

Modelele de tip ANOVA sunt acele modele de regresie liniară în care toate variabilele explicative (independente) sunt exclusiv de natură calitativă, fiind exprimate prin variabile dummy. În acest scenariu, scopul regresiei nu mai este modelarea unei pante de creștere continuă, ci compararea mediilor variabilei dependente Y între diferite subgrupuri ale populației. În cel mai simplu caz, în care se studiază impactul unei singure variabile atributive cu două stări (de exemplu, genul angajatului: Femeie sau Bărbat), modelul ANOVA se formulează astfel: $y_i = \beta_0 + \beta_1 D_i + u_i$. Unde $D_i = 1$ dacă angajatul este bărbat și 0 dacă este femeie. Valoarea așteptată (speranța matematică) a variabilei dependente pentru fiecare grup se deduce prin condiționarea erorii:

- Pentru categoria codificată cu 0 (Femeie): $E(y_i | D_i = 0) = \beta_0$
- Pentru categoria codificată cu 1 (Bărbat): $E(y_i | D_i = 1) = \beta_0 + \beta_1$

Această descompunere evidențiază rolul crucial al parametrilor într-un model ANOVA. Termenul liber (β_0) nu mai este doar un punct teoretic de intersecție cu axa ordonatelor, ci reprezintă valoarea medie a variabilei dependente pentru categoria de referință (baza de comparație). Coeficientul pantei artificiale (β_1) măsoară diferența absolută dintre media grupului analizat și media grupului de referință. Testarea ipotezei nule $H_0: \beta_1 = 0$ devine echivalentă cu testarea existenței unei diferențe semnificative din punct de vedere statistic între cele două medii, oferind o punte matematică directă între analiza de regresie și testul statistic tradițional de comparație a mediilor.

În majoritatea aplicațiilor economice practice, fenomenele sunt mult prea complexe pentru a fi explicate doar prin atribute calitative.

Modelele de tip ANCOVA extind analiza prin îmbinarea simultană a variabilelor cantitative (continue) cu variabile calitative (dummy). În această arhitectură, variabilele cantitative standard poartă denumirea de covariabile sau variabile de control. Un model ANCOVA clasic, care analizează salariul (y) în funcție de o covariabilă cantitativă (vechimea în muncă, x) și o variabilă dummy (genul, D), îmbracă următoarea formă deterministă la nivelul eșantionului: $y_i = \beta_0 + \beta_1 x_i + \beta_2 D_i + u_i$. Dacă analizăm speranța matematică a acestui model pentru fiecare subgrup determinat de variabila dummy, obținem două ecuații de regresie distincte:

- Pentru categoria de referință ($D_i = 0$): $E(y_i | X_i, D_i = 0) = \beta_0 + \beta_1 x_i$
- Pentru categoria analizată ($D_i = 1$): $E(y_i | X_i, D_i = 1) = (\beta_0 + \beta_2) + \beta_1 x_i$

Din punct de vedere geometric și econometric, modelul ANCOVA descrie două linii de regresie (sau hiperplane în cazul regresiei multiple) care sunt perfect paralele, având exact aceeași pantă (β_1), dar intercepturi diferite. Coeficientul β_1 arată efectul marginal pur al variabilei cantitative (cu cât crește, în medie, salariul pentru fiecare an de vechime suplimentar, menținând genul constant). Coeficientul β_2 acționează ca un efect de translație pe verticală: el indică diferența constantă de salariu dintre cele două categorii de angajați, indiferent de nivelul vechimii acestora.

Studiul bibliografic al modelelor ANOVA și ANCOVA relevă faptul că integrarea variabilelor dummy permite extinderea ariei de aplicabilitate a modelului liniar multiplu la o gamă vastă de scenarii din viața reală. Flexibilitatea acestor modele rezidă în capacitatea lor de a izola efectele structurale și calitative, oferind cercetătorului un instrument analitic puternic pentru controlul eterogenității din seturile de date brute.

2.2.2. Regula celor $m-1$ variabile și analiza teoretică a coliniarității perfecte (Capcana Variabilelor Dummy)

Transformarea variabilelor calitative în regresori numerici prin intermediul variabilelor dummy reprezintă o soluție econometrică elegantă, însă implementarea ei necesită respectarea unor reguli algebrice stricte pentru a asigura stabilitatea modelului. Problema fundamentală care apare în această etapă de modelare este legată de numărul exact de variabile artificiale care trebuie generate pentru a reprezenta o caracteristică. În literatura de specialitate, codificarea stărilor unei variabile atributive este guvernată de o axiomă

structurală: dacă o variabilă calitativă posedă un număr de m categorii (stări) distincte și mutual exclusive, atunci în ecuația de regresie se vor introduce exact $m-1$ variabile dummy.

Categoria care este lăsată intenționat în afara modelului nu dispăre din analiză, ci devine starea de referință (sau categoria de bază). Din punct de vedere matematic, starea de referință este definită de situația în care toate cele $m-1$ variabile dummy introduse iau simultan valoarea 0. Importanța acestei categorii este crucială în faza de interpretare a rezultatelor, deoarece coeficienții asociați variabilelor dummy incluse în model nu reprezintă valori absolute, ci măsoară diferența (abaterea) față de această categorie de referință.

Transpusă în arhitectura modelului matriceal $Y = X\beta + u$, această situație provoacă o colapsare a sistemului de ecuații normale. Matricea de design X conține întotdeauna pe prima sa coloană un vector format exclusiv din cifra 1, necesar pentru estimarea termenului liber (interceptului, β_0). Dacă se introduc toate cele m variabile dummy, suma coloanelor acestora va genera, pe fiecare rând, exact valoarea 1. Prin urmare, coloana interceptului poate fi obținută ca o combinație liniară perfectă a coloanelor variabilelor dummy:

$$\text{Coloana}_{\text{Intercept}} = \text{Coloana}_{D_1} + \text{Coloana}_{D_2} + \dots + \text{Coloana}_{D_m}$$

Din punct de vedere algebric, existența unei combinații liniare perfecte între coloanele matricei X înseamnă că rangul matricei nu mai este maxim. Aceasta reprezintă o încălcare flagrantă a uneia dintre ipotezele fundamentale Gauss-Markov: ipoteza absenței multicolinearității perfecte.

Consecința directă este că produsul matriceal $X^T X$ devine o matrice singulară (o matrice cu determinantul zero). Întrucât ecuația fundamentală de calcul a estimatorilor OLS necesită inversarea acestei matrice: $\hat{\beta} = (X^T X)^{-1} X^T Y$ algoritmul matematic eșuează. Inversa unei matrice singulare nu există (operația fiind echivalentă cu o împărțire la zero în algebra scalară), iar coeficienții de regresie devin nedeterminați. Astfel, "**Capcana Variabilelor Dummy**" nu reprezintă doar o eroare teoretică de interpretare, ci un defect structural fatal care împiedică rezolvarea sistemului și obținerea soluției econometrice. Eliminarea celei de-a m -a categorii rupe această liniaritate perfectă, restabilind independența coloanelor matricei X și permițând rezolvarea numerică a modelului.

2.3. Analiză Numerică Computațională și Descompunerea LU

Transpunerea ecuației teoretice a estimatorilor OLS, $\hat{\beta} = (X^T X)^{-1} X^T Y$, într-un mediu de dezvoltare software aduce în prim-plan provocări specifice analizei numerice computaționale. Deși din punct de vedere algebric inversarea matricei $X^T X$ este o operație directă și clară, în practica informatică calculul explicit al inversei este considerat ineficient și extrem de predispus la erori de precizie. Arhitecturile de calcul reprezintă numerele reale folosind standarde în virgulă mobilă (precum IEEE 754), ceea ce induce erori inerente de rotunjire. În cazul matricelor de dimensiuni mari sau al celor „aproape singulare” (situații în care multicoliniaritatea între variabile este ridicată, dar nu perfectă), aceste erori microscopice se propagă exponențial pe parcursul operațiilor matriceale, conducând la fenomenul de instabilitate numerică. Mai mult, algoritmul clasic de inversare a unei matrice complet singulare (cum este cazul declanșării „capcanei variabilelor dummy”) va eșua silențios sau va genera erori fatale de execuție, blocând întregul proces de analiză a datelor.

Pentru a eluda aceste limitări computaționale, ecosistemele software moderne și bibliotecile de calcul științific evită calculul explicit al inversei, recurgând la metode numerice avansate și stabile pentru rezolvarea sistemelor de ecuații liniare. Cea mai recunoscută și utilizată procedură în acest sens este **Descompunerea LU** (Lower-Upper). Principiul matematic al acestei metode constă în factorizarea unei matrice pătratice A (care, în cazul sistemului de ecuații normale, este matricea $X^T X$ în produsul a două matrice triunghiulare: $A = L \cdot U$).

În această identitate, L (Lower) reprezintă o matrice inferior triunghiulară, care are elemente nenule doar pe diagonala principală și sub aceasta, în timp ce U (Upper) este o matrice superior triunghiulară, cu elemente nenule exclusiv deasupra diagonalei. Odată realizată această factorizare, sistemul original de ecuații normale, $(X^T X)\beta = X^T Y$, poate fi rescris sub forma $L(U\beta) = B$ (unde B este vectorul cunoscut $X^T Y$). Această reconfigurare permite rezolvarea sistemului în două etape computaționale simple, bazate pe substituție:

1. Se rezolvă ecuația $Ly = B$ prin substituție directă (de sus în jos) pentru a determina vectorul intermediar y .
2. Se rezolvă ecuația $U\beta = y$ prin substituție inversă (de jos în sus) pentru a extrage vectorul final al coeficienților estimați, $\hat{\beta}$.

Din perspectivă informatică, implementarea descompunerii LU reduce drastic complexitatea temporală (numărul total de operații efectuate de procesor) și optimizează

consumul resurselor de memorie. Abordarea LU se remarcă prin robustețea sa structurală: previne degradarea datelor prin erori de reprezentare și evaluează mult mai precis determinanții, oprind execuția controlat atunci când detectează o matrice singulară. Astfel, aplicarea acestui algoritm numeric garantează acuratețea absolută a estimărilor econometrice, permițând aplicațiilor software să gestioneze modele cu zeci de variabile calitative și cantitative fără a compromite integritatea matematică a rezultatelor.

2.4. Tehnologii moderne pentru automatizarea mediilor de calcul

2.4.1. Evoluția soluțiilor de automatizare în foile de calcul: de la limbajul VBA la ecosistemul web modern Office.js

Aplicațiile de calcul electronice, în special Microsoft Excel, au încetat de mult să fie simple tabele pentru stocarea datelor, devenind veritabile medii de dezvoltare și platforme de calcul. Pe măsură ce volumul de date și complexitatea modelelor matematice au crescut, necesitatea automatizării fluxurilor de lucru repetitive și a extinderii funcționalităților native a determinat o evoluție profundă a tehnologiilor de programare integrate în aceste aplicații Office.

Din perspectivă istorică, pilonul central al automatizării în Microsoft Excel a fost reprezentat, timp de aproape trei decenii, de limbajul VBA (Visual Basic for Applications), introdus la începutul anilor '90. Bazat pe arhitectura COM (Component Object Model), VBA a revoluționat productivitatea corporativă și academică, permițând utilizatorilor să scrie macrocomenzi, să manipuleze direct structura celulelor și să construiască interfețe grafice complexe (UserForms) integrate nativ în registrele de calcul (.xlsm). Execuția codului VBA se realiza local, direct în procesul aplicației gazdă, oferind un acces rapid la resursele memoriei ram și o interacțiune de latență joasă cu interfața Excel.

Cu toate acestea, tranziția societății informaționale către era Cloud Computing-ului, a mobilității și a securității cibernetice stricte a scos la iveală limitările structurale severe ale paradigmei VBA, determinând deprecierea treptată a acesteia în proiectele software moderne. Codul VBA rulează cu drepturi de execuție depline pe mașina utilizatorului. Din acest motiv, fișierele Excel cu macrocomenzi au devenit, din punct de vedere istoric, unul dintre cei mai folosiți vectori de atac pentru răspândirea de malware, troieni și ransomware. Politicile moderne de securitate din companii blochează adesea execuția macrocomenzilor, limitând utilitatea soluțiilor bazate pe acest limbaj. VBA a fost proiectat pentru sistemele de operare desktop (în principal Windows). Odată cu diversificarea ecosistemului de

dispozitive, codul VBA s-a dovedit complet incompatibil cu versiunile mobile de Excel (iOS, Android) și, cel mai important, cu versiunea Excel on the Web (Cloud). Chiar și pe sistemele macOS, suportul pentru VBA a fost istoric fragmentat și incomplet. VBA utilizează un model de execuție strict sincron și pe un singur fir de execuție (single-threaded). În momentul în care un algoritm econometric complex execută operații matriceale grele, întreaga interfață a aplicației Excel îngheață (devine unresponsive), alterând grav experiența utilizatorului. Limbajul VBA nu a mai primit actualizări majore de sintaxă sau arhitectură de mulți ani. El este complet izolat de instrumentele moderne de inginerie software, cum ar fi sistemele de versionare (Git), managerii de pachete (npm) sau framework-urile moderne de testare și design web.

Pentru a răspunde acestor provocări specifice erei internetului, Microsoft a dezvoltat o nouă paradigmă de extensibilitate: Office Add-ins, având la bază API-ul JavaScript unificat cunoscut sub numele de Office.js. Această mutație tehnologică reprezintă trecerea de la un limbaj de scriptare proprietar și local (VBA) la utilizarea tehnologiilor web standard deschise: HTML5, CSS3 și JavaScript/TypeScript.

Arhitectura modernă bazată pe Office.js aduce o serie de avantaje esențiale pentru dezvoltarea aplicațiilor de calcul științific din noua generație:

1. Un Add-in dezvoltat cu Office.js este rulat în mod identic pe orice platformă. Utilizatorul poate lansa aplicația în Excel pentru Windows, Excel pentru Mac, pe o tabletă Android/iPad sau direct într-un browser web (Excel on the Web), fără a modifica o singură linie de cod.
2. Spre deosebire de VBA, un add-in web rulează într-un container securizat izolat (sandbox), similar unui tab dintr-un browser modern. Aplicația comunică cu Excel exclusiv prin intermediul unui canal API securizat și nu are acces direct la sistemul de fișiere local sau la memoria sistemului de operare, eliminând riscul infectării cu viruși.
3. Interacțiunea cu foaia de calcul este complet asincronă, bazată pe concepte moderne precum Promises și sintaxa async/await. Prin intermediul metodei fundamentale Excel.run, operațiile de citire și scriere a datelor sunt plasate într-o coadă de execuție gestionată eficient în fundal. Interfața Excel rămâne fluidă și activă chiar și în timpul rulării unor calcule econometrice de anvergură.
4. Folosind JavaScript, dezvoltatorii pot integra instant biblioteci de calcul de înaltă performanță (cum ar fi mathjs sau jstat), framework-uri de interfață grafică avansate (React, Vue) și stilizări moderne.

În concluzie, evoluția de la abordarea monolitică a VBA la ecosistemul modular și extensibil al Office.js marchează o aliniere a automatizării analizei datelor la standardele

globale ale ingineriei software actuale. Această tranziție facilitează crearea unor instrumente econometrice puternice, sigure și accesibile direct din cloud, transformând foaia de calcul dintr-un simplu spațiu de stocare local într-o aplicație distribuită și interactivă.

2.4.2. Managementul preciziei datelor în medii runtime asincrone (reprezentarea numerelor în JavaScript)

Adoptarea ecosistemului web (prin API-ul Office.js) pentru automatizarea modelelor econometrice aduce provocări profunde la nivelul arhitecturii de calcul matematic. Deși limbajul JavaScript a devenit un standardul global pentru dezvoltarea de aplicații asincrone, motorul său nativ de procesare numerică nu a fost proiectat inițial pentru calcul științific sau algebră liniară de înaltă fidelitate. Înțelegerea și depășirea limitărilor acestui motor sunt pași critici pentru validarea oricărui instrument statistic dezvoltat în acest mediu.

Spre deosebire de limbajele de programare tradiționale (cum ar fi C++ sau Java), care oferă o varietate de tipuri de date numerice (întregi de diferite mărimi, numere reale cu simplă sau dublă precizie), JavaScript implementează, prin standardul ECMAScript, un singur tip de dată numeric de bază: tipul *Number*. Acesta este reprezentat exclusiv în format binar cu virgulă mobilă în dublă precizie pe 64 de biți (Double Precision 64-bit floating-point), conform standardului internațional IEEE 754. Din cei 64 de biți alocați în memorie pentru un număr, 1 bit este rezervat pentru semn, 11 biți definesc exponentul, iar restul de 52 de biți reprezintă mantisa (fracția sau precizia efectivă). Această limitare structurală generează o problemă fundamentală în analiza numerică: incapacitatea de a reprezenta exact anumite fracții zecimale în format binar. De exemplu, un calcul banal precum $0.1 + 0.2$ nu va evalua exact la 0.3 în memoria procesorului, ci la 0.30000000000000004.

În aplicațiile web uzuale, aceste anomalii microscopice de rotunjire (floating-point rounding errors) sunt neglijabile. Totuși, în contextul algebrei matriceale aplicate în econometrie, impactul lor devine periculos. Evaluarea modelului OLS presupune operații matriciale masive: înmulțirea matricei transpuse cu matricea originală $X^T X$, calcularea sumei pătratelor reziduurilor sau descompunerea LU. Atunci când se ridică la pătrat valori financiare mari (ex. salarii sau PIB) în paralel cu valori foarte mici (ex. rate ale dobânzii sub 1%), se declanșează un fenomen computațional numit catastrofă de anulare (catastrophic cancellation). Erorile de rotunjire de la nivelul zecimalelor se propagă și se amplifică exponențial la fiecare nouă operație de înmulțire matriceală. Cea mai gravă consecință a

acestui fenomen în modelarea liniară o reprezintă falsa singularitate: o matrice care matematic are un determinant foarte aproape de zero (dar nenul) poate fi evaluată de procesor ca având determinantul zero din cauza pierderii de precizie, provocând blocarea algoritmului. Invers, o matrice teoretic singulară (cauzată de capcana variabilelor dummy) poate căpăta un determinant microscopic diferit de zero, determinând algoritmul să genereze estimatori $\hat{\beta}$ complet falși, dar care par valizi în interfață.

Pentru a atinge același nivel de rigoare științifică precum instrumentele comerciale (EViews, SPSS sau Data Analysis Toolpak din Excel) rulând în același timp într-un mediu asincron web-based, este imperativă abandonarea tipului nativ *Number* în favoarea unor structuri de date capabile de aritmetică cu precizie arbitrară. Literatura de inginerie software recomandă utilizarea bibliotecilor matematice de înaltă precizie (precum *mathjs*), care introduc tipul de date *BigNumber*. Spre deosebire de numerele standard, un obiect de tip *BigNumber* nu folosește instrucțiunile hardware ale procesorului (ALU - Arithmetic Logic Unit) destinate standardului IEEE 754. În schimb, numărul este stocat în memoria aplicației sub forma unui vector (array) de cifre sau ca șir de caractere (string), alocând memorie dinamic în funcție de numărul de zecimale necesare. Operațiile matematice (adunări, înmulțiri matriceale, inversări) sunt executate prin algoritmi software la nivel de caracter. Deși această abstractizare implică un consum suplimentar de resurse și un timp de procesare ușor mai ridicat, într-un mediu asincron acest lucru nu afectează reactivitatea interfeței cu utilizatorul. Beneficiul absolut este eliminarea completă a erorilor de rotunjire. Prin utilizarea tipurilor *BigNumber*, se garantează că matricea informațională $X^T X$ își păstrează exactitatea teoretică absolută, asigurând calculul corect al erorilor standard, al testelor de semnificație (*p-values*) și validitatea inferenței econometrice, transformând astfel ecosistemul web într-o platformă robustă pentru cercetarea cantitativă.

CAPITOLUL 3. PROIECTARE DE DETALIU ȘI IMPLEMENTAREA SISTEMULUI SOFTWARE

3.1. Arhitectura Detaliată și Fluxul de Date al Aplicației

3.1.1. Structura modulară a Add-in-ului

Proiectarea unei aplicații software destinate calculului științific și econometric în cadrul unui ecosistem preexistent, cum este Microsoft Excel, impune adoptarea unei arhitecturi riguroase, ghidată de principiul fundamental al Separării Preocupărilor (Separation of Concerns - SoC). În dezvoltarea Add-in-urilor web bazate pe API-ul Office.js, acest principiu devine o necesitate tehnică. Deoarece un Add-in web este, în esență, o aplicație web care rulează într-un container izolat (un mediu de tip webview/iframe) în interiorul aplicației gazdă Excel, cuplarea strânsă a logicii de prezentare cu algoritmul matematic ar conduce la un cod rigid, greu de întreținut și dificil de testat.

Pentru a asigura o structură robustă, extensibilă și stabilă, arhitectura sistemului implementat a fost divizată în trei module independente, clar delimitate din punctul de vedere al responsabilităților:

1. **Stratul de prezentare (User Interface Layer):** modul care gestionează exclusiv interfața grafică vizibilă de către utilizator în panoul lateral de lucru (Taskpane). Construit folosind standardele HTML5 și CSS3, stratul UI se ocupă de randarea elementelor de control (butoane, meniuri de selecție a modelului econometric, câmpuri de configurare) și de managementul stării locale a interfeței. Responsabilitatea sa principală este să captureze intenția utilizatorului și să preia selecțiile făcute în foaia de calcul.
2. **Stratul de conectare al datelor (Office.js Connector):** acționează ca un intermediar asincron între interfața utilizatorului, foaia de calcul Excel și motorul de calcul. Acest modul utilizează API-ul Office.js pentru a citi valorile brute din celulele selectate de utilizator și pentru a scrie rezultatele înapoi în registrele Excel. Rolul său critic este de a asigura transformarea structurală: el preia matricile de celule din Excel (care conțin adesea șiruri de caractere sau amestecuri de date) și le serializează sub formă de tablouri bidimensionale pure, gata de a fi procesate de nucleul matematic. De asemenea, tot acest strat interceptează erorile de sistem și asigură că execuția asincronă nu blochează interfața grafică.

3. **Motorul Econometric și Matematic (Core Math Engine):** reprezintă nucleul computațional al aplicației și este complet izolat de ecosistemul Microsoft Office sau de elementele de interfață. Scris în JavaScript pur, acest modul nu are nicio dependență față de API-ul Office.js sau față de DOM-ul (Document Object Model) paginii web. El primește ca intrări exclusiv structuri de date matematice abstracte (tablouri bidimensionale de numere sau obiecte de tip BigNumber) și returnează un obiect JSON cu metricile rezultate (coeficienți, erori standard, valori p-value, R^2 , statistici t și F). Izolarea acestui motor permite utilizarea optimă a librăriilor specializate precum mathjs și jstat pentru execuția descompunerii LU.

Această structură modulară oferă avantaje tehnice majore pentru calitatea aplicației software dezvoltate. Deoarece motorul econometric este izolat și nu depinde de contextul Excel, toți algoritmi de calcul (inclusiv gestionarea variabilelor dummy și regresia OLS) pot fi testați complet automatizat în afara browserului sau a aplicației Excel, prin transmiterea unor matrici de test și verificarea output-ului. Erorile apărute în motorul matematic (cum ar fi detectarea unei matrice singulare în timpul descompunerii LU) sunt prinse (catch) și transformate în mesaje structurate, trimise prin data bridge direct către UI. Acest lucru previne prăbușirea (crash-ul) întregului add-in și permite afișarea unui mesaj de avertizare elegant pentru utilizator. acă pe viitor se dorește adăugarea unui nou model econometric (de exemplu, o regresie logistică) sau modificarea designului interfeței grafice, dezvoltatorul poate lucra izolat pe modulul respectiv, având certitudinea că modificările nu vor altera stabilitatea celorlalte componente.

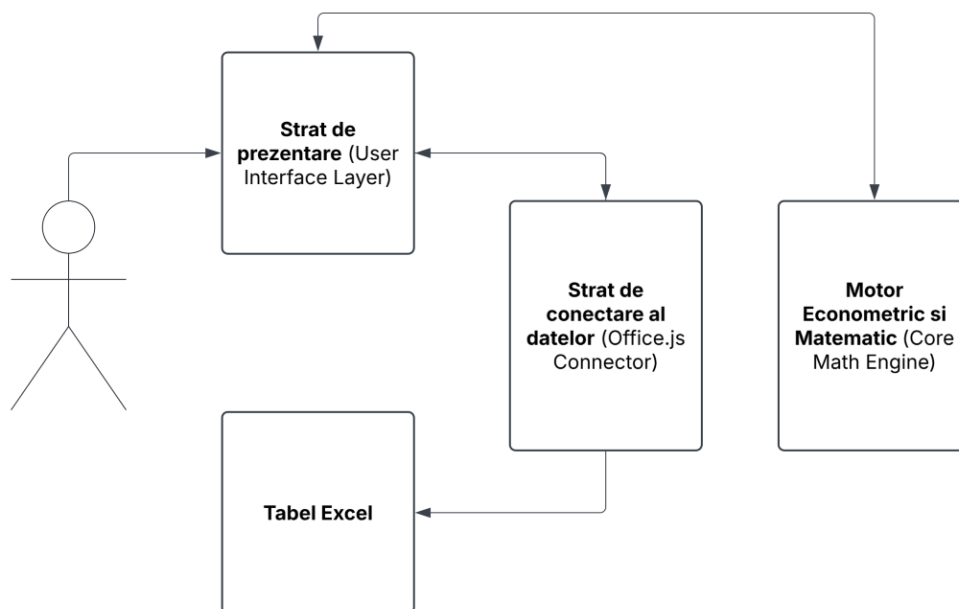


Figura 3.1. Diagrama conceptuală și componentele add-in-ului
Sursa: Prelucrare proprie

3.1.2. Fluxul de date

Pentru a înțelege modul în care aplicația transformă un set de date brut selectat de utilizator dintr-o foaie de calcul într-un raport econometric complex și complet structurat, este necesară analizarea detaliată a pipeline-ului de execuție. Acest flux de date (Data Pipeline) urmează o logică strict secvențială și asincronă, fiind guvernat de evenimentele declanșate în interfața grafică și procesate în straturile de integrare și calcul descrise anterior. Fluxul complet parcurge șase etape fundamentale, descrise în detaliu în cele ce urmează:

1. Intercepția evenimentului și capturarea contextului (UI -> Office.js): Pipeline-ul este inițiat în momentul în care utilizatorul finalizează selecția celulelor ce conțin variabilele econometrice direct în foaia de calcul Excel și apasă pe butonul de execuție a analizei din panoul lateral (*taskpane.html*). În acest punct, componenta UI trimite o cerere asincronă către stratul de conectare. Prin intermediul metodei unificate *Excel.run(async (context) => {...})*, add-in-ul interoghează obiectul *context.workbook.getSelectedRange()* pus la dispoziție de API-ul Office.js. Această operație capturează metadatele selecției: coordonatele spațiale ale celulelor, numărul de rânduri și coloane, precum și referințele către antetele de tabel (numele variabilelor).

2. Extragerea asincronă și maparea datelor brute: Deoarece operațiile de citire din Excel sunt asincrone pentru a preveni blocarea interfeței, se apelează metoda *.load("values")*, urmată de sincronizarea contextului prin *await context.sync()*. În acest moment, datele celulelor fizice din Excel intră în memoria de lucru a add-in-ului sub forma unui tablou bidimensional nativ de tip text/număr.

3. Pre-procesarea și codificarea automată a variabilelor dummy: Datele brute extrase sunt direcționate imediat către modulul de pre-procesare din logica de programare. Aici, pipeline-ul execută o rutină critică de scanare a coloanelor. Funcția dedicată identifică coloanele ce conțin date de tip text (variabile calitative) și inițiază algoritmul de binarizare. Pentru o variabilă calitativă cu m stări detectate, algoritmul generează automat $m-1$ coloane noi umplute strict cu valori de 0 și 1, eliminând în mod controlat o categorie pentru a preveni capcana variabilelor dummy. Tot în această etapă, în funcție de modelul selectat de utilizator (Liniar sau Log-Liniar), se aplică transformările logaritmice asupra variabilelor cantitative continue. La finalul acestei etape, datele sunt convertite în întregime în obiecte de înaltă precizie de tip *BigNumber* pentru a asigura imunitatea la erorile de rotunjire.

4. Construirea matricelor de design și transferul către motorul econometric: în acest punct al pipeline-ului, datele pre-procesate sunt structurate conform algebrei liniare

analizate în studiul bibliografic. Se generează matricea de design X , unde prima coloană este populată artificial cu valoarea 1 (pentru intercept), urmată de coloanele variabilelor cantitative și cele ale variabilelor dummy rezultate din codificare și vectorul coloană Y , corespunzător variabilei dependente. Aceste două structuri matematice abstracte sunt transmise prin apel direct către funcția principală a nucleului econometric, izolată de orice interfață: *calculateRegression(yData, xData, modelType)*.

5. Execuția algoritmului OLS prin descompunere LU: motorul econometric preia controlul fluxului și execută operațiile matriceale în cascadă. Se calculează transpusa X^T , urmată de produsele $X^T X$ și $X^T Y$. În loc să calculeze inversa explicită, motorul apelează rutina stabilă de analiză numerică *math.lusolve(X_T_X, X_T_Y)* pentru a determina vectorul de coeficienți $\hat{\beta}$. Ulterior, o buclă iterativă calculează vectorul valorilor prezise $\hat{Y}$, vectorul reziduurilor $u = Y - \hat{Y}$ și sumele pătratelor (SST, SSR, SSE). Folosind o metodă bazată pe vectori coloană elementari, algoritmul extrage doar elementele de pe diagonala principală a inversei necesare pentru determinarea erorilor standard, apelând în final funcțiile de repartiție din librăria jstat pentru a deduce statisticile t, F și valorile aferente de probabilitate (p-values). Toate aceste rezultate sunt împachetate într-un obiect de tip JSON structurat.

6. Generarea raportului formatat și scrierea în foaia de calcul: obiectul JSON rezultat revine prin puntea de integrare către stratul de prezentare. Stratul de prezentare solicită stratului de conectare să insereze în fișierul Excel datele calculate anterior.

Prin acest pipeline bine delimitat, aplicația asigură un transfer sigur, rapid și fără pierderi de precizie de la celula brută din Excel la raportul final interpretat, automatizând complet un proces care, în mod tradițional, necesita un efort manual considerabil.

3.2. Implementarea algoritmilor în JavaScript

3.2.1. Dezvoltarea modului de pre-procesare a datelor

Pentru ca motorul econometric să poată evalua corect sistemul de ecuații prin algebra matriceală, matricea de design X trebuie să conțină exclusiv valori numerice. Cu toate acestea, în practica analizei datelor, utilizatorii selectează frecvent coloane din Excel care conțin variabile calitative exprimate sub formă de text (de exemplu: „Urban”/„Rural”, niveluri de educație, gen etc.). Pentru a face legătura între forma brută a datelor și rigoarea cerută de algoritmul OLS, a fost necesară proiectarea unui modul software intermediar de conversie. Această responsabilitate este încapsulată în funcția de transformare, denumită

preprocessDummyVariables. Rolul acestei funcții este de a acționa ca un filtru inteligent: analizează setul de date independent (matricea X), detectează automat prezența coloanelor textuale și le convertește în variabile numerice binare (dummy), aplicând strict regulile econometrice definite în capitolele teoretice.

Logica funcției a fost structurată pe un algoritm iterativ în patru etape principale: scanarea coloanelor și detecția tipului de date, identificarea și extragerea categoriilor unice, aplicarea regulii m-1 și stabilirea categoriei de referință, ultimul pas fiind binarizarea și restructurarea matricei.

Modulul de pre-procesare scanează inițial matricea datelor extrase pentru a identifica automat coloanele de tip text (variabilele atributive). Pentru fiecare coloană textuală detectată, algoritmul extrage și reține doar valorile unice (categoriile distincte), determinând astfel numărul total de stări (m) ale acelei variabile. În etapa de conversie, pentru a preveni blocajele matematice precum „Capcana Variabilelor Dummy” (coliniaritatea perfectă), sistemul aplică regula m-1 prin excluderea automată a unei categorii, care devine baza de referință a modelului. Pentru restul categoriilor reținute, algoritmul restructurează matricea adăugând noi coloane binare, în care atribuie valoarea 1 dacă textul original corespunde categoriei respective, sau 0 în caz contrar.

După generarea tuturor coloanelor dummy necesare, lângă coloana textuală originală este/sunt inserată/e coloana/ele generate prin identificarea și separarea valorilor coloanei dummy. În același timp, funcția actualizează și vectorul cu metadate (numele variabilelor), combinând numele coloanei originale cu numele stării binarizate (ex: "Studii_Master", "Studii_Doctorat"), astfel încât stratul de interfață (UI) să poată afișa denumiri explicite în tabelul final de rezultate. Utilizatorul apoi poate adăuga coloanele generate în foaia de lucru a modelului principal (dacă este nevoie).

Prin implementarea funcției *preprocessDummyVariables*, aplicația automatizează o procedură laborioasă și predispusă la erori umane. Utilizatorul este degrevat de necesitatea de a crea manual coloane suplimentare cu formule condiționale de tip IF în Excel pentru codificarea datelor. Motorul econometric primește la final o matrice numerică validă, imună la singularități induse de structura datelor, pregătită direct pentru descompunerea LU și calculul estimatorilor.

3.2.2. Programarea motorului econometric OLS

Transpunerea nucleului matematic al OLS într-un modul software executabil și stabil în JavaScript necesită o atenție deosebită în ceea ce privește alegerea funcțiilor algoritmice și gestionarea structurilor de date. În cadrul aplicației dezvoltate, întreaga această logică computațională este încapsulată în funcția principală *calculateRegression*. Pentru a garanta imunitatea algoritmului la erorile de rotunjire specifice reprezentării standard în virgulă mobilă, motorul econometric utilizează exclusiv tipurile de date de înaltă precizie *BigNumber* oferite de biblioteca *mathjs*. Cu toate acestea, provocarea majoră în faza de programare nu a fost reprezentată de stocarea numerelor, ci de depășirea limitărilor structurale și a instabilităților specifice funcțiilor de algebră liniară la manipularea matricelor complexe.

În faza inițială de dezvoltare, o abordare naivă bazată pe calculul explicit al inversei prin intermediul funcției clasice *math.inv(X_T_X)* s-a dovedit a fi extrem de vulnerabilă. Pe lângă erorile masive de propagare numerică la seturi mari de date, încercarea de a optimiza algoritmul prin trecerea directă la descompunerea LU a lovit o barieră de design a bibliotecii externe. La încercarea de a calcula inversa matricei informaționale prin rezolvarea simultană a sistemului cu o matrice identitate completă, runtime-ul JavaScript a declanșat o eroare fatală de tipul: *RangeError: Dimension mismatch. Matrix columns must match vector length*.

Această eroare apare deoarece funcția *math.lusolve(A, b)* din *mathjs* este proiectată structural pentru a rezolva un sistem liniar în care argumentul *b* este un singur vector coloană (o structură unidimensională sau o matrice cu o singură coloană). În momentul în care i se transmite o matrice multidimensională completă în locul unui vector, algoritmul intern de substituție își pierde consistența dimensională și blochează execuția.

Pentru a depăși acest impas computațional și pentru a asigura o implementare robustă, arhitectura motorului econometric a fost reproiectată pentru a evita complet calculul integral al inversei, exploatând matematic proprietățile vectorilor coloană elementari. Soluția software implementată în cod rezolvă ecuațiile în doi pași distincți și stabili:

1. Determinarea vectorului de coeficienți ($\hat{\beta}$): întrucât produsul $X^T Y$ generează în mod nativ un vector coloană de dimensiune $(k+1) \times 1$, ecuația normală $(X^T X)\beta = X^T Y$ se

încadrează perfect în constrângerile de design ale funcției *math.lusolve*. Codul execută operația:

```
// 1. Aflăm coeficienții direct (Beta = X_T_X \ X_T_Y)
// X_T_Y este un vector coloană, deci lusolve funcționează perfect
const BetaRaw = math.lusolve(X_T_X, X_T_Y);
const Beta = BetaRaw.toArray ? BetaRaw.toArray() : BetaRaw;
```

Figura 3.2. Cod specific determinării vectorului de coeficienți
Sursa: Prelucrare proprie

Această secvență apelează direct descompunerea LU în fundal, returnând vectorul coeficienților $\hat{\beta}$ (interceptul și pantele) cu o precizie absolută, eliminând riscul degradării zecimalelor chiar și în prezența unor numere de ordinul sutelor de mii.

2. Calculul selectiv al erorilor standard (se) prin vectori elementari: din punct de vedere econometric, obținerea erorilor standard ale coeficienților necesită extragerea elementelor aflate strict pe diagonala principală a inversei $(X^T X)^{-1}$. Pentru a obține aceste elemente fără a inversa direct întreaga matrice (ceea ce ar fi declanșat eroarea de dimensiune), s-a dezvoltat o buclă iterativă. Pentru fiecare variabilă i (de la 0 la k), algoritmul construiește dinamic un vector coloană elementar e_i , populat cu valori de 0 de tip BigNumber, cu excepția poziției i care primește valoarea 1. Acest vector reprezintă o singură coloană din matricea identitate virtuală. Sistemul este apoi rezolvat individual.

```
// Creăm vectorul coloană e_i (plin cu 0, și 1 la poziția i)
const e_i = [];
for (let j = 0; j < k + 1; j++) {
  e_i.push([math.bignumber(i === j ? 1 : 0)]);
}

// Rezolvăm X_T_X * v_i = e_i (acum trimitem strict un vector, eroarea dispare)
const v_i_raw = math.lusolve(X_T_X, e_i);
const v_i = v_i_raw.toArray ? v_i_raw.toArray() : v_i_raw;

// Elementul de pe diagonală este elementul "i" din vectorul calculat
const diagElement = Array.isArray(v_i[i]) ? v_i[i][0] : v_i[i];
```

Figura 3.3. Cod specific determinării erorilor
Sursa: Prelucrare proprie

Deoarece e_i este un vector coloană pur, funcția *math.lusolve* rulează la eficiență maximă și fără erori dimensionale. Elementul de pe poziția i din vectorul rezultat v_i corespunde exact elementului de pe diagonală căutat, $((X^T X)^{-1})_{ii}$. Ulterior, valoarea este

multiplicată cu pătratul erorii standard a estimării (ESE^2), iar prin extragerea radicalului se obține eroarea standard precisă a coeficientului respectiv.

Această strategie de programare aduce avantaje majore performanței software a add-in-ului. Pe de o parte, economisește resurse substanțiale de procesare, deoarece elimină calculele inutile pentru elementele din afara diagonalei inversei (care nu sunt folosite în testele statistice). Pe de altă parte, oferă o stabilitate numerică excepțională: dacă utilizatorul introduce date eronate care forțează o coliniaritate perfectă, algoritmul de descompunere LU detectează imediat singularitatea în mod controlat și permite aruncarea unei excepții personalizate către UI, prevenind blocarea sau prăbușirea aplicației Excel. Rezultatul este un motor econometric stabil, capabil să asigure alinierea perfectă cu rezultatele oferite de pachetele statistice desktop dedicate.

3.2.3. Logica de transformare dinamică a datelor în funcție de tipul modelului (liniar, log-liniar)

Un principiu fundamental al analizei de regresie este că relația dintre variabile nu este întotdeauna liniară. Fenomenele economice prezintă frecvent creșteri proporționale, elasticități constante sau rate de variație care se atenuează în timp, tipare care nu pot fi surprinse printr-un model liniar simplu. Pentru a acomoda această diversitate, aplicația implementează patru specificații de model: linear, log-linear, semi-log și lin-log, fiecare corespunzând unei ipoteze distincte despre natura relației dintre variabile.

Din punct de vedere matematic, toate cele patru specificații se reduc, în final, la același algoritm OLS, cu diferența că sunt aplicate pe date transformate. Prin aplicarea logaritmului natural asupra variabilei dependente, a celor independente, sau a ambelor, se „liniarizează” relații intrinsec neliniare, permițând estimarea prin metoda celor mai mici pătrate clasice. Această abordare este corectă din perspectiva ipotezelor Gauss-Markov: liniaritatea este cerută în parametri, nu în variabilele originale.

Transformarea este implementată în funcția `transformDataForModel`. Logica de decizie este compactă și simetrică:

```
const transformY = modelType === "log-linear" || modelType === "semi-log";  
const transformX = modelType === "log-linear" || modelType === "lin-log";
```

Figura 3.4. Cod logica de decizie a transformării datelor

Sursa: Prelucrare proprie

Cele două flag-uri booleene controlează independent transformarea fiecărei axe, generând cele patru combinații posibile:

Tabel 3.1. Logica de decizie a transformării datelor

Model	$\ln(Y)$	$\ln(X)$	Interpretare coeficient β
Linear	x	x	variație absolută
Log-linear	✓	✓	elasticitatea
Semi-log	✓	x	rata de creștere
Lin-log	x	✓	variație absolută la %

Sursa: Prelucrare proprie

Transformarea este aplicată element cu element, utilizând tipul `bignumber` din biblioteca `mathjs` pentru a evita erorile de precizie în virgulă mobilă, specifice JavaScript. Dacă oricare valoare este nulă sau negativă, funcția aruncă o eroare explicită, deoarece logaritmul nu este definit pe $\mathbb{R}^-$. Vectorul Y transformat și matricea X augmentată cu coloana de 1 (pentru intercept) sunt returnate și transmise direct motorului de calcul OLS. Astfel, modulul de transformare acționează ca un strat de preprocesare transparent: utilizatorul selectează tipul de model din interfață, iar restul se derulează automat, fără a fi necesară nicio intervenție manuală asupra datelor brute.

3.3. Proiectarea UI și a modului de interpretare automată

3.3.1. Designul contextual al panoului de lucru (Taskpane HTML/CSS) și gestiunea stărilor în UI

Dezvoltarea interfeței grafice a presupus abordarea unor provocări specifice ecosistemului Microsoft Office, unde aplicația rulează ca un Office Add-in integrat într-un container de tip `webview` sau `iframe` direct în interiorul aplicației Excel. Punctul de intrare în interfață este reprezentat de un document HTML minimalist, `taskpane.html`, care funcționează ca un shell de bază. Acesta conține exclusiv un element `div` cu identificatorul `container`, destinat injectării întregului arbore de componente React, și scriptul `office.js` necesar pentru apelarea API-urilor native. Tot la acest nivel a fost implementat un mecanism de siguranță care detectează motoarele de randare învechite, precum Trident sau Edge Legacy, afișând un mesaj de incompatibilitate înainte de a încărca logica aplicației. Decizia arhitecturală majoră în faza de proiectare a fost abandonarea fișierelor CSS clasice în favoarea paradigmei CSS-in-JS. Stilizarea se realizează exclusiv prin intermediul bibliotecii `@fluentui/react-components` (Fluent UI React v9) și a sistemului său `makeStyles`. Această

alegere asigură o încapsulare perfectă a stilurilor, previne coliziunile de clase CSS și garantează o coerență vizuală nativă cu restul interfeței Microsoft 365.

Structura vizuală principală este orchestrată de componenta rădăcină *App.jsx*, care utilizează un layout bazat pe flexbox vertical, constrâns la o lățime maximă de 500 de pixeli pentru a optimiza afișarea în panoul lateral îngust. Interfața este divizată logic în trei zone funcționale clare. Zona superioară, *topPart*, acționează ca un antet ce conține titlul și subtitlul aplicației, fiind delimitată vizual de restul conținutului printr-o linie de bordură subtilă. Zona centrală, *middlePart*, este configurată să ocupe flexibil spațiul disponibil și găzduiește mecanismul principal de navigare. Zona inferioară, *bottomPart*, funcționează ca un subsol minimalist, conținând un buton cu o pictogramă de setări care deschide modalul de configurare globală. Pentru a păstra rigoarea alinierilor, distanțelor și culorilor, toate aceste zone sunt stilizate apelând token-uri de design predefinite din ecosistemul Fluent UI, precum *tokens.spacingHorizontalM* sau *tokens.colorNeutralStroke3*.

Navigarea către modulele de analiză econometrică este gestionată printr-un pattern de tip declanșator-dialog. În zona centrală a aplicației sunt randate patru butoane de tip *ActionButton*, corespunzătoare modulelor de Statistici Descriptive, Regresie, Comparare Modele și Variabile Dummy. Componenta *ActionButton* ocupă întreaga lățime a containerului, având o înălțime minimă care facilitează interacțiunea tactilă, și integrează o pictogramă colorată specifică modulului în partea stângă, textul descriptiv, urmat de un indicator vizual sub forma unei săgeți de navigare în partea dreaptă. Aceste butoane acționează ca elemente de tip *DialogTrigger*. La interacțiune, ele lansează în prim-plan o componentă modală de tip *Dialog*, cum ar fi *ModalSimpleRegression.jsx*. Arhitectura fiecărui modal respectă un șablon strict, fiind compus dintr-un titlu, o zonă de conținut pentru configurare și o zonă de acțiuni cu butoane pentru validare sau anulare. Zona de conținut permite utilizatorului să definească tipul modelului dorit printr-o componentă *Combobox*, să introducă nivelul de semnificație statistică și să activeze sau să dezactiveze interpretarea econometrică automată printr-un comutator de tip *Switch*. Starea locală a acestor formulare este menținută eficient prin intermediul hook-ului *useState* din React, asigurând actualizarea instantanee a interfeței. Tot la acest nivel, conținutul textual este expus utilizatorului apelând hook-ul custom *useLanguage*, care expune funcția *t* pentru a extrage dinamic traduceri din dicționarele în limba română sau engleză.

Piesa centrală a interacțiunii dintre utilizator și foaia de calcul este componenta personalizată *RangeSelector*, inclusă în formularele modale pentru a defini variabilele dependente și independente. Această componentă este compusă dintr-un câmp de

introducere a textului și un buton de acțiune atașat. Rolul ei este de a sincroniza interfața web cu mediul nativ Excel. Când utilizatorul apasă butonul, componenta execută un apel asincron *Excel.run()*, care interoghează motorul Office.js pentru a prelua adresa exactă a celulelor selectate în acel moment în foaia de calcul. Adresa este apoi extrasă și injectată automat în starea locală a aplicației React, populând câmpul de text. Acest flux elimină necesitatea introducerii manuale a coordonatelor, reducând considerabil fricțiunea și riscul de eroare la definirea modelului econometric.

Dincolo de stările locale ale formularelor, aplicația necesită o coordonare globală a preferințelor utilizatorului, cerință soluționată prin implementarea *SettingsContext.jsx*. Acest context React acționează ca un depozit central, ținut în memoria volatilă a browser-ului pentru sesiunea curentă, și gestionează trei parametri fundamentali: tema vizuală curentă, numărul de zecimale pentru precizia rapoartelor generate și nivelul de verbozitate al textelor explicative, denumit modul de interpretare. Componentele inferioare accesează și modifică această stare prin intermediul funcției expuse și a hook-ului dedicat *useSettings*.

Pentru a optimiza logica de randare a rapoartelor econometrice, arhitectura include și un hook derivat, *useInterpretation*. Acesta extrage modul curent de lucru și expune variabile booleene și metode utilitare, precum funcția *fillFor*. Această metodă decide dinamic dacă o anumită celulă din raportul final Excel trebuie evidențiată cromatic, returnând un obiect de fundal specific dacă modul educațional este activ, sau o valoare nulă dacă utilizatorul a optat pentru un raport compact.

Pentru a oferi o identitate vizuală distinctă, dar familiară utilizatorilor platformei, întregul sistem a fost ancorat într-o tematică definită în fișierul *theme.js*. Baza o constituie o paletă de brand personalizată, *dizenBrand*, derivată din culoarea verde caracteristică aplicației Excel, extinsă algoritmic în zece trepte de luminozitate. Folosind funcțiile utilitare din Fluent UI, paleta a fost utilizată pentru a genera două teme complete, *dizeLightTheme* și *dizeDarkTheme*, care suprascriu la nivel global token-urile de culoare pentru suprafețe, borduri și text. Mai mult, estetica este rafinată prin impunerea fontului sans-serif 'Libre Franklin' pe întregul arbore HTML și prin aplicarea unei raze de curbă uniformă de patru pixeli pentru toate elementele interactive. Rezultatul este o interfață modernă, responsive și perfect asimilată vizual în ecosistemul de productivitate al Microsoft.

3.3.2. Algoritm de generare dinamică a textelor de interpretare econometrică în funcție de pragurile de semnificație (p-values)

În mod tradițional, instrumentele software dedicate analizei statistice (inclusiv funcția Data Analysis Toolpak din Excel) furnizează rezultatele exclusiv sub forma unor tabele matriceale dense, încărcate cu valori numerice brute. Deși corect din punct de vedere matematic, acest format impune o sarcină cognitivă ridicată asupra utilizatorului, necesitând cunoștințe econometrice avansate pentru extragerea concluziilor practice. Pentru a transforma aplicația dintr-un simplu calculator într-un asistent educațional și decizional, a fost conceput un modul de interpretare automată. Acesta poate fi activat opțional printr-un comutator (*Switch*) din interfața utilizatorului. Când este activ, algoritmul nu doar că rezolvă ecuațiile, ci traduce rezultatele numerice într-un raport narativ, scris în limbaj natural direct în foaia de calcul, adaptându-se dinamic la metadatele datelor și la deciziile de modelare.

Arhitectura acestui modul se bazează pe o funcție centrală, denumită *interpretationRegression*, care primește ca argumente vectorii cu statisticile calculate, metadatele variabilelor selectate de utilizator (numele seriilor extrase din Excel), nivelul de semnificație ales (α) și instanța de traducere pentru suportul multilingv. Execuția algoritmului urmează un flux secvențial structurat pe trei etape logice (pași):

Pasul 1: Construcția ecuației modelului

În primă fază, algoritmul generează expresia algebrică a modelului estimat. Acest proces este delegat funcției *getEquation*, care analizează tipul de regresie ales de utilizator. Algoritmul construiește dinamic un șir de caractere de forma $Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k$. Prin evaluarea tipurilor log-liniare, semi-logaritmice sau lin-log, funcția aplică contextual prefixul matematic $\ln()$ exclusiv asupra variabilelor supuse transformării, oferind utilizatorului reprezentarea vizuală corectă a formei funcționale modelate. Tot în acest pas, dacă modul educațional este activ, se afișează sinteza indicatorilor globali de performanță (valorile p-value și R^2).

Pasul 2: Evaluarea semnificației statistice

Cea de-a doua etapă reprezintă nucleul testării ipotezelor.

Funcția *getMultipleRegressionSignificance* iterează prin fiecare coeficient de pantă calculat (β_j) și evaluează validitatea statistică a acestuia prin apelul funcției *getSignificance*. Pentru fiecare variabilă independentă, algoritmul formulează automat ipoteza nulă ($H_0: \beta_j = 0$).

Pentru a asigura o acoperire metodologică exhaustivă, algoritmul nu evaluează indicatorul p-value exclusiv față de pragul α selectat din interfață. Funcția auxiliară *buildAlphaSet* generează o matrice de comparație care conține valorile clasice consacrate în literatură (0.01, 0.05, 0.10), la care adaugă, sortat crescător, nivelul personalizat al utilizatorului, dacă acesta diferă. Ulterior, funcția *getConclusion* analizează relația matematică dintre p-value și fiecare element al acestui set, returnând unul dintre cele trei verdicte predefinite:

Tabel 3.2.

Semnificativ	Atunci când p-value este strict mai mic decât pragul α .
Nesemnificativ	Atunci când p-value depășește pragul α .
Marginal (Edge-case)	O inovație algoritmică importantă o reprezintă capabilitatea de a detecta situațiile în care valoarea p-value se află la limita superioară a pragului de decizie (diferența fiind mai mică de 0.02). În acest scenariu, funcția generează un text de avertizare, atenționând utilizatorul că variabila este „marginal semnificativă” și că decizia de a o păstra în model comportă riscuri.

Sursa: Prelucrare proprie

Textul final generat pentru fiecare concluzie este extras din fișierele de localizare (i18n), interpolând dinamic valorile testelor și numele variabilelor, garantând fluiditatea gramaticală a rapoartelor. În cazul modelelor cu regresori multipli, acest pas este urmat de testarea globală de tip Fisher (F-test), construită pe baza aceluiași algoritm iterativ multi-nivel.

Pasul 3: Interpretarea economică a coeficienților estimați

Ultima etapă, orchestrată prin funcția *getInterpretation*, convertește estimările matematice în concepte tangibile. Aceasta parcurge vectorul pantelor și generează pentru fiecare regresor o propoziție care explică impactul său asupra variabilei dependente. Algoritmul adaptează unitățile de măsură la specificul modelului: în cazul modelelor log-liniare sau lin-log, creșterea marginală a variabilei X este exprimată textual sub forma unui procent („1%”), iar coeficientul (β_j) este înmulțit algoritmic cu 100 pentru a reflecta rata de creștere procentuală.

Un rafinament deosebit intervine în tratarea variabilelor atributive (Dummy). Dacă proprietatea *isDummy* a metadatelor este evaluată ca fiind adevărată, algoritmul schimbă complet paradigma textului: în loc să analizeze variația continuă, textul generat va compara categoria curent analizată (codificată cu 1) cu categoria de referință a modelului (codificată cu 0, lăsată intenționat în afara matricei), reflectând astfel perfect teoria capcanei

variabilelor dummy expusă anterior. Secțiunea este finalizată prin interpretarea indicatorului R^2 ajustat, tradus textual sub forma procentajului din variația fenomenului explicat de model.

Pentru a consolida asocierea cognitivă dintre tabelele masive de date și propozițiile explicative, interfața web și motorul econometric colaborează printr-un sistem avansat de mapare cromatică. Fiecare statistică generată primește o culoare unică prin funcția *randColor*. Acest algoritm calculează un hash deterministic pe baza valorii numerice și a identificatorului variabilei, pe care îl mapează apoi pe roata culorilor (spațiul HSL, menținând saturația la 60 și luminozitatea la 83% pentru a asigura un contrast optim pe fundalul alb). Rezultatul este un cod hexazecimal unic. Funcția derivată *fillFor* evaluează modul de afișare selectat global din *SettingsContext*: dacă aplicația se află în modul educațional (STUDENT), codul de culoare este injectat direct în API-ul Excel ca formatare de umplere a celulelor. Astfel, valoarea numerică a unui p-value din tabel va avea exact aceeași nuanță de fundal ca și cuvântul „semnificativ” din propoziția corespondentă. Dacă utilizatorul preferă o abordare sobră (COMPACT), funcția *fillFor* returnează o valoare nulă, anulând formatarea cromatică și reducând textul generat strict la verdictul final. Această adaptabilitate poziționează soluția software la granița dintre un instrument analitic puternic și o platformă pedagogică interactivă.

3.4. Testare, Validare și Analiză Comparativă (Validare Empirică)

3.4.1. Strategia de testare a algoritmului propriu pe seturi de date cu număr variabil de regresori

Deoarece nucleul matematic al aplicației a fost proiectat pentru a gestiona dinamic modele cu arhitecturi diferite, strategia de testare și validare nu s-a bazat pe scenarii automate de tip unit-testing (lipsind fișierele *.test.js*), ci pe o validare empirică riguroasă, executată manual pe seturi de date reale introduse direct în foile de calcul Excel. Această abordare iterativă a permis observarea directă a modului în care algoritmul reacționează la modificările de structură și format ale datelor de intrare, testând limitele matematice ale funcției *calculateRegression*.

Procesul de validare a urmat o traiectorie progresivă în ceea ce privește numărul de regresori (k). Testele inițiale au vizat regresia liniară simplă ($k=1$), verificând dacă algoritmul populează corect matricea de design X cu dimensiunea $n \times 2$ (o coloană pentru intercept și una pentru variabila independentă). Ulterior, complexitatea a fost crescută treptat

către modele de regresie multiplă cu doi, trei și peste cinci regresori. Aceste teste au demonstrat că arhitectura software scalează impecabil: funcția de preprocesare detectează automat numărul k interogând lungimea rândului extras (*xData.data[0].length*), generând dinamic matricea augmentată $n \times (k+1)$ și extinzând automat calculul erorilor standard și al Factorului de Inflație a Varianței (VIF) pentru fiecare variabilă suplimentară adăugată, fără a necesita intervenția sau reconfigurarea manuală a parametrilor din partea utilizatorului.

O a doua dimensiune a testării a vizat flexibilitatea structurală a modelării matematice. S-au rulat iterativ seturi de date pentru toate cele patru forme funcționale suportate de interfață: liniar, log-liniar, semi-logaritmic și lin-log. Obiectivul a fost verificarea funcției *transformDataForModel*. Rezultatele au confirmat că algoritmul aplică corect transformarea prin logaritm natural ($\ln$) asupra variabilei dependente Y sau a vectorilor X , iar modulul de interpretare a preluat aceste modificări, generând textual ecuația corespunzătoare (ex: $\ln(Y) = \dots$). În paralel, s-a validat detecția inteligentă a variabilelor calitative (dummy). Scanând matricea, dacă o coloană conține exclusiv valori de 0 și 1, algoritmul setează automat atributul *isDummy* pe valoarea true. Testele au dovedit că modulul de generare a textelor recunoaște acest flag și modifică sintaxa propozițiilor, făcând referire directă la "categoria de referință", validând astfel teoria econometrică implementată.

În final, s-a testat robustețea sistemului de raportare și configurare. Modificarea pragului de semnificație (α) către valori atipice sau stricte (precum 0.01 sau 0.10) a demonstrat că funcția *buildAlphaSet* recalibrează dinamic testele ipotezelor, recalculând simultan limitele intervalelor de încredere prin intermediul funcției inverse *jStat.studentt.inv*. Trecerea interfeței din modul detaliat (STUDENT) în cel restrâns (COMPACT) a confirmat că raportul Excel generat ascunde textele narative extinse și anulează injectarea culorilor de formatare în celule, oferind un output tehnic condensat, perfect pentru analiza profesională rapidă.

3.4.2. Auditarea rezultatelor: analiză comparativă și confruntarea zecimalelor cu modulul oficial Microsoft (Data Analysis Toolpak)

Validitatea absolută a oricărui instrument econometric software este dată de acuratețea matematică a estimărilor sale. Pentru a audita performanța și precizia algoritmului OLS dezvoltat, s-a recurs la o analiză comparativă (benchmark) cu aplicația consacrată din industrie: Data Analysis Toolpak, extensia nativă oferită de Microsoft Excel.

Comparația a vizat setul complet de metrici de ieșire, de la estimările coeficienților (β) și erorile lor standard, până la statisticile testelor (valori t, F, p-value și coeficienții de determinație (R^2 , R^2 ajustat)).

Din perspectivă arhitecturală, cele două aplicații operează fundamental diferit, ceea ce face ca auditul să fie deosebit de relevant pentru evidențierea calității codului implementat. *Data Analysis Toolpak* utilizează motorul matematic tradițional al Excel-ului, fundamentat pe standardul hardware IEEE 754 (reprezentare în virgulă mobilă cu dublă precizie pe 64 de biți), capabil să asigure o precizie de aproximativ 15-16 zecimale semnificative. Mai mult, algoritmul Microsoft recurge la inversarea explicită a matricei $(X^T X)^{-1}$, o operație recunoscută în literatura de specialitate pentru instabilitatea sa în cazul matricelor complexe.

În contrast, aplicația dezvoltată introduce o inovație computațională majoră. Motorul matematic este construit pe tipul de date *BigNumber* (furnizat de biblioteca *mathjs*), configurat global în *mathConfig.js* pentru a susține o precizie extremă de 64 de zecimale exacte în timpul rulării, evitând pierderile de informație în etapele intermediare. Mai mult, pentru a asigura stabilitatea, algoritmul eludează calculul inversei explicite. Sistemul liniar $X^T X * \beta = X^T Y$ este rezolvat prin descompunere LU (apelând *math.lusolve*). Pentru extragerea erorilor standard, nucleul matematic rezolvă iterații succesive ale sistemului $X^T X * v_i = e_i$ (unde e_i reprezintă vectorul canonic), extrăgând direct elementul necesar de pe diagonală $v_i[i]$.

La confruntarea directă a rezultatelor pe seturi de date economice bine condiționate, s-a constatat o identitate perfectă a rezultatelor. Deoarece funcția custom *toUINumber* rotunjește automat output-ul la 4 zecimale pentru afișare, diferențele invizibile de precizie (dincolo de zecimala a 15-a) sunt trunchiate uniform, output-ul vizibil în interfața aplicației fiind matematic identic (0.00% eroare) cu cel generat de Toolpak-ul Microsoft.

Diferențele apar însă în favoarea aplicației dezvoltate atunci când sunt introduse date cu un grad ridicat de coliniaritate (matrice aproape singulare). În aceste condiții, Toolpak-ul are tendința de a forța inversarea, generând fie coeficienți complet instabili (fără sens economic), fie populând raportul cu erori fatale de tip *#NUM!*. În schimb, arhitectura bazată pe descompunerea LU absoarbe șocul matematic, detectează singularitatea și oprește execuția controlat.

În plus, auditul a reliefat superioritatea funcțională a noii aplicații: pe lângă rapoartele narrative, instrumentul dezvoltat furnizează metrici avansate, complet absente din

Toolpak, precum coeficienții standardizați (Beta weights), matricea de corelație a predictorilor și factorul VIF pentru diagnosticul multicolinearității, oferind un pachet de analiză net superior din perspectiva exhaustivității.

3.4.3. Gestionarea situațiilor limită în execuție (edge-cases) și tratarea erorilor

Pentru ca un produs software să poată fi utilizat în producție, el trebuie să fie rezilient la erori de operare umană sau la anomalii în datele de intrare. Aplicația dezvoltată a fost proiectată cu mecanisme defensive clare, capabile să intercepteze și să gestioneze elegant (fără prăbușirea platformei) șase situații limită critice (edge-cases) identificate în timpul fazei de testare empirică.

Cea mai severă excepție matematică este apariția unei matrice singulare, generată de obicei prin declanșarea capcanei variabilelor dummy sau prin introducerea unor predictorii perfect coliniari. În această situație, funcția *math.lusolve* eșuează instantaneu, refuzând pe bună dreptate să rezolve un sistem nedeterminat. Deși eroarea este aruncată la nivelul nucleului *calculateRegression*, absența unui bloc *try-catch* local permite propagarea ei naturală în lanțul de apeluri asincrone, până în funcția de interfațare cu Excel-ul (*getColumnMatrix*). Aici, sistemul global de prindere a excepțiilor captează evenimentul, returnând un tablou gol (*[]*) care oprește silențios execuția. Astfel, utilizatorul este protejat de rapoarte false. A doua restricție fundamentală vizează transformările logaritmice. Funcția *transformDataForModel* validează strict elementele matricei înainte de aplicarea logaritmului natural. Dacă algoritmul întâlnește o valoare egală cu 0 sau negativă ($v \leq 0$), el aruncă o excepție dedicată (*throw new Error(...)*), specificând clar dacă problema se află în vectorul dependent sau în cel independent (ex: „Logaritmul necesită valori strict pozitive pentru Y”). Mesajul ajunge în UI, permițând utilizatorului să corecteze setul de date.

Mecanisme de filtrare inteligentă au fost introduse și la extragerea datelor brute. Dacă selecția din Excel este goală, funcția *getColumnMatrix* returnează o matrice nulă, întrerupând calculele inutile. Mai mult, dacă utilizatorul selectează accidental rânduri care conțin text (în afară de antet) sau celule goale intercalate printre valori numerice, algoritmul le ignoră silențios. Rândurile viciate nu sunt incluse în matricea de design, salvând execuția, în timp ce logurile de diagnosticare consemnează evenimentul pentru dezvoltatori. În aceeași etapă de parsare a interfeței, codul verifică dacă primul rând este compus exclusiv din șiruri de caractere (*typeof cellValue = "string"*). În caz afirmativ, rândul este automat promovat ca antet (metadata) ce reține numele variabilelor, iar dacă formatul conține

paranteze unghiulare (ex: „Salariu <RON>”), algoritmul izolează separat unitatea de măsură. Dacă antetul lipsește, algoritmul consideră primul rând ca făcând parte din setul de date efectiv.

Ultimele două optimizări abordează detaliile fine ale raportării. Funcția de formare a output-ului vizual, *toUINumber*, integrează o ramură logică specială (dacă modulul numărului < 0.001) prin care evită trunchierea automată la 0.0000 a valorilor foarte mici. Această excepție garantează că indicatorii critici, precum p-value-urile puternic semnificative (ex: 0.000312), sunt afișate integral, protejând sensul statistic. Tot pentru eficientizarea resurselor computaționale, algoritmul detectează instanța regresiei simple ($k=1$). Într-un model cu un singur regresor, conceptul de coliniaritate nu are sens matematic; așadar, funcția de diagnosticare ignoră iterațiile masive de calcul pentru diagnosticul VIF, atribuind direct valoarea 1 parametrului, stabilizând astfel performanța pentru modelele de dimensiuni reduse.

CAPITOLUL 4. MANUAL DE UTILIZARE AL APLICAȚIEI

4.1. Cerințe tehnice și compatibilitate

Aplicația software a fost proiectată pentru a asigura o compatibilitate vastă cu ecosistemul modern Microsoft, rulând nativ ca un Office Add-in. Din punct de vedere al mediului de operare, instrumentul oferă suport complet pentru versiunile desktop de Excel pentru Windows (începând cu Microsoft 365 și Office 2021) și Excel pentru Mac (Microsoft 365). O trăsătură tehnică definitorie a arhitecturii sale web-based este funcționarea identică, fără a necesita vreo modificare a codului sursă, direct în browser prin platforma Excel on the Web (office.com). Pe de altă parte, aplicația este incompatibilă cu versiunile mobile (iOS și Android), cu edițiile Office mai vechi de 2019 și cu versiunile care utilizează motoare de randare învechite, precum Trident (Internet Explorer) sau EdgeHTML (Edge Legacy). În aceste scenarii neacceptate, interfața detectează automat limitarea tehnică a browserului și blochează execuția, afișând utilizatorului un mesaj explicit care recomandă actualizarea versiunii de Office.

În faza de dezvoltare și testare, instalarea add-in-ului presupune o procedură de încărcare manuală (sideloading). Aceasta necesită existența mediului Node.js pe mașina locală pentru a porni serverul de dezvoltare, precum și generarea unui certificat SSL local pentru a asigura conexiunea securizată pe localhost. Integrarea efectivă în Excel se realizează prin încărcarea fișierului manifest.xml din meniul aplicației. Pentru a funcționa corect, platforma necesită exclusiv permisiunea de tip ReadWriteDocument, acordată la nivel de manifest, care permite algoritmului să citească valorile din celulele selectate de utilizator și să scrie rezultatele formate în destinațiile indicate.

Din perspectiva stivei tehnologice relevante pentru performanța resimțită de utilizatorul final, aplicația este construită pe platforma React 19, folosind biblioteca Fluent UI React 9 pentru o interfață perfect aliniată estetic cu suita Office. Calculele matematice complexe sunt asigurate de bibliotecile mathjs versiunea 15 (utilizând exclusiv tipul de date BigNumber pentru o precizie extinsă) și jStat pentru extragerea distribuțiilor statistice. Un avantaj major de securitate și performanță este faptul că întreaga logică de calcul rulează strict local, în memoria calculatorului sau a browserului, fără a transfera datele financiare sau academice ale utilizatorului către servere externe pe internet.

4.2. Ghid de utilizare pas cu pas

4.2.1. Pregătirea și structurarea datelor

Corectitudinea analizei econometrice depinde fundamental de modul în care datele sunt introduse în foaia de calcul. Utilizatorul trebuie să organizeze informațiile în coloane verticale continue, valorile numerice fiind dispuse pe rânduri corespundente fiecărei observații. Deși includerea unui antet de tabel este opțională, utilizarea acestuia este puternic recomandată. Algoritmul de preprocesare analizează automat primul rând al selecției; dacă acesta conține exclusiv text, este marcat instantaneu ca antet, iar denumirile sunt preluate pentru a personaliza rapoartele și textele de interpretare. În absența unui antet text, variabilele vor primi denumiri generice, precum X1, X2 și Y. Mai mult, aplicația suportă o convenție avansată pentru definirea unităților de măsură: dacă antetul include o unitate încadrată între paranteze unghiulare (de exemplu, „Salariu <RON>”), sistemul va separa automat denumirea de unitate, utilizându-le contextual în propozițiile din raportul final. Pe parcursul selecției, prezența unor rânduri cu date lipsă sau text accidental nu va bloca algoritmul, acestea fiind filtrate și ignorate silențios.

IDU <puncte>	CGE <mil USD>	CGS <mil USD>
0.715	1166	1560
0.722	1556	1660
0.729	1837	1882
0.732	2106	2038
0.742	2724	2425
0.756	3587	3228
0.77	5047	4019
0.786	6724	6363
0.806	9168	7821
0.809	6658	7023
0.807	5534	6899
0.808	7557	7621
0.805	5109	6585
0.81	5354	7684
0.811	5969	7983
0.813	5447	7474
0.816	6224	7609
0.823	6016	9153
0.827	7637	11238
0.832	9108	12500
0.824	9190	13760
0.821	9816	15260

Figura 4.1. Model date corecte de intrare
Sursa: Curs Econometrie + prelucrare proprie

Pentru analizele cantitative, cum ar fi modelele de regresie și extragerea statisticilor descriptive, datele trebuie să fie strict numerice. O restricție matematică deosebit de importantă intervine în cazul selectării unor specificații logaritmice (modele log-liniare, semi-logaritmice sau lin-log). În aceste situații, toate valorile seriilor de date ce urmează a fi transformate trebuie să fie strict pozitive. În cazul în care algoritmul întâlnește valori zero sau negative, va întrerupe execuția. Pentru analizarea datelor calitative prin intermediul modului de variabile dummy, coloana originală din Excel trebuie să conțină texte care definesc categoriile (de exemplu: „Masculin” / „Feminin” sau „Urban” / „Rural”). Aplicația va scana automat această coloană, va detecta toate stările unice și le va afișa în interfață, permițând utilizatorului să selecteze direct starea de referință (baseline), moment în care instrumentul va injecta rapid în foaia de calcul restul coloanelor binare (m-1) necesare analizei.

4.2.2. Configurarea și lansarea analizei

Interacțiunea principală cu instrumentul se realizează prin intermediul panoului lateral (Taskpane), care devine vizibil imediat după lansarea aplicației din meniul Excel. Acest panou centralizează patru butoane de comandă principale, corespunzătoare modulelor de calcul: Statistici Descriptive, Regresie Simplă și Multiplă, Comparare Modele și Variabile Dummy. În subsolul panoului se regăsește iconița de acces către setările globale ale aplicației. Pentru a lansa o regresie, utilizatorul accesează formularul dedicat, unde trebuie să specifice coordonatele datelor. Selectarea rangurilor în componentele de interfață se poate face fie prin scrierea manuală a adresei din Excel, fie mult mai facil, prin efectuarea selecției direct pe foaia de calcul urmată de apăsarea butonului-iconiță atașat câmpului, acțiune care captează și injectează automat adresa curentă. Utilizatorul va defini variabila dependentă (Y) printr-o singură coloană, în timp ce variabilele independente (X) pot fi selectate simultan prin evidențierea mai multor coloane alăturate, aplicația deducând automat numărul de regresori (k). Din același formular se alege destinația raportului, tipul modelului dorit (dintr-un meniu derulant ce include variantele Liniar, Log-Liniar, Semi-Log și Lin-Log) și nivelul de semnificație (α), care este preconfigurat la 0.05, dar poate fi ajustat.

Un element central al interfeței este comutatorul pentru activarea interpretării econometrice, care transformă output-ul dintr-un simplu tabel cu cifre într-un text narativ explicativ.

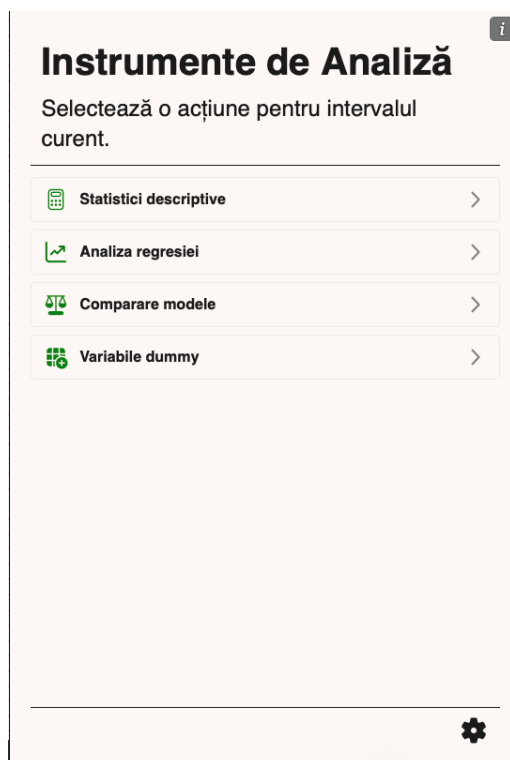


Figura 4.2. Meniu principal (Taskpane)

Sursa: Prelucrare proprie

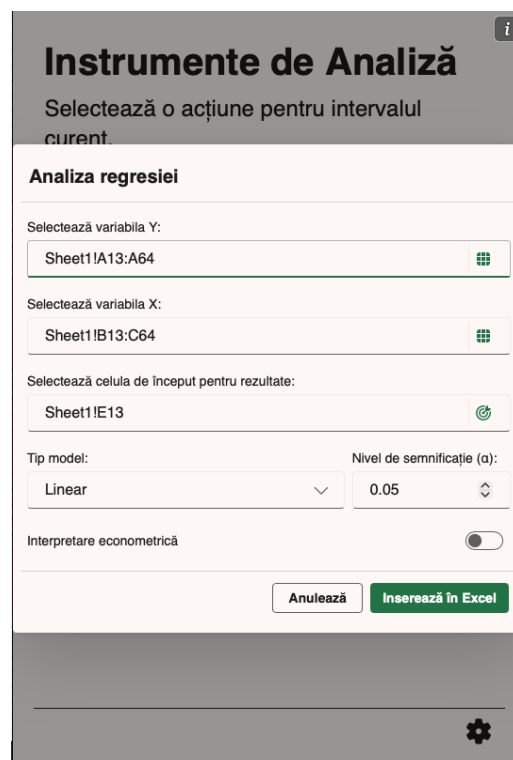


Figura 4.3. Modal selecție date pentru analiza regresiei

Sursa: Prelucrare proprie

Modulul de Comparare Modele oferă un flux de lucru avansat, necesitând selectarea a cel puțin două specificații din listă. Odată lansat, acesta execută paralel algoritmi OLS pentru fiecare tip de model selectat pe același set de date și expune un raport consolidat, util pentru cercetători. Modulul de Variabile Dummy propune un flux ghidat în doi pași: selectarea coloanei textuale și a celulei de destinație, urmată de selectarea explicită a categoriei de referință prin intermediul unor butoane de tip radio, finalizându-se prin comanda de generare. Comportamentul platformei poate fi rafinat accesând meniul de setări globale. De aici, utilizatorul poate schimba limba interfeței (Română sau Engleză), tema vizuală (Light sau Dark) și nivelul de afișare a zecimalelor (între 2 și 64). Cel mai important parametru global este Modul de Interpretare, unde se poate opta între modul „Student”, care este detaliat și folosește un sistem avansat de codare vizuală a rezultatelor, și modul „Compact”, optimizat pentru a livra concluzii rapide și concise.

4.2.3. Citirea și interpretarea rapoartelor generate

Odată ce analiza a fost executată, aplicația transcrie rezultatele direct în foaia de calcul sub forma unui raport complet și formatat profesional. Structura vizuală a acestui

raport este împărțită logic în șase secțiuni distincte, ușor de parcurs datorită paletelor cromatice care utilizează nuanțe de verde preluate din identitatea vizuală a aplicației Excel. Prima secțiune prezintă statisticile globale ale regresiei (precum numărul de observații, R^2 și Eroarea Standard), urmată imediat de tabelul ANOVA. A treia secțiune, tabelul coeficienților, reprezintă inima raportului, detaliind pentru fiecare variabilă estimarea matematică, erorile standard, statisticile testului t, valoarea probabilității și limitele intervalului de încredere, incluzând suplimentar și coeficienții standardizați (beta weights). Dacă utilizatorul a optat pentru generarea textelor narrative (având switch-ul activ și modul „Student” selectat din setări), va observa a patra secțiune a raportului: blocul de interpretare. Acesta va lista ecuația estimată a modelului și va testa separat fiecare coeficient pentru a oferi verdicte clare privind semnificația, luând în calcul nu doar nivelul ales de utilizator, ci și pragurile clasice de 0.01, 0.05 și 0.10. De asemenea, va furniza o traducere economică a fiecărei pante calculate. În cazul modelelor de regresie multiplă ($k > 1$), va apărea automat a cincea secțiune, care evaluează riscul de multicolinearitate analizând matricea de corelație și factorul VIF. Raportul se încheie cu o „Concluzie Generală”, o concluzie generală inteligentă care sintetizează într-o singură frază performanța globală a modelului și identifică cel mai influent factor pe baza coeficienților standardizați.

În modul de învățare (Student), citirea raportului este facilitată de un sistem de codare prin culori. Orice valoare numerică din tabelul coeficienților primește o culoare de fundal unică, generată determinist. Aceeași nuanță este aplicată pe celula de text din secțiunea de interpretare corespunzătoare aceluși număr. Astfel, utilizatorul poate naviga vizual și instantaneu între o statistică brută și propoziția care îi explică semnificația, accelerând masiv procesul de înțelegere.

Dacă utilizatorul bifează opțiunea „Compact” din setări, aspectul și destinația raportului se schimbă. Acesta devine o concluzie generală minimalistă, fără culori de fundal, ascunzând pașii intermediari ai testelor și afișând doar verdictele finale scurte. În acest mod, pentru a proteja integritatea foilor de calcul complexe, raportul nu va fi inserat lângă date, ci va fi scris automat pe o nouă filă Excel dedicată (precum *Discussion_Regression*), generând astfel un document curat, ideal pentru a fi exportat sau inclus direct într-o prezentare profesională.

CAPITOLUL 5. CONCLUZII, LIMITĂRI ȘI DIRECȚII VIITOARE

5.1. Concluzii privind obiectivele realizate

Prezenta lucrare de disertație a demonstrat cu succes fezabilitatea și utilitatea integrării tehnologiilor web moderne în automatizarea proceselor de analiză econometrică tradițională. Obiectivul central al cercetării a fost eliminarea barierei tehnice și de timp pe care o întâmpină analiștii, studenții și profesorii, eliminând procesele repetitive de analiză a datelor. Prin proiectarea și implementarea acestui Office Add-in personalizat, s-a reușit transpunerea unui flux de lucru matematic extrem de complex într-o soluție software intuitivă, capabilă să livreze rezultate validate științific în doar câteva interacțiuni (click-uri) din partea utilizatorului. Arhitectura modulară dezvoltată a dovedit că limitările istorice ale limbajului VBA pot fi depășite elegant prin utilizarea ecosistemului Office.js combinat cu React JS. S-a realizat o decuplare eficientă între interfața grafică, construită fluent cu *@fluentui/react-components*, și nucleul computațional izolat. Algoritmii implementați automatizează etape care, în mod tradițional, erau predispușe la eroare umană: de la recunoașterea și codificarea automată a variabilelor atributive (dummy) cu aplicarea strictă a regulii categoriei de referință, până la transformările logaritmice dinamice ale datelor brute. Mai mult, inovația adusă de modulul de interpretare narativă a transformat aplicația dintr-un simplu calculator matriceal într-un veritabil asistent educațional. Generarea dinamică a textelor în limbaj natural, corelată vizual prin sistemul de culori deterministe cu pragurile de semnificație statistică, validează pe deplin scopul practic al lucrării, oferind utilizatorului nu doar cifre, ci și o înțelegere profundă a fenomenului economic modelat.

5.2. Limitări Tehnice și Econometrice Identificate

Deși soluția software implementată rezolvă o gamă vastă de probleme de modelare liniară, arhitectura sa și natura matematicii pe care se fundamentează impun anumite limitări inerente, care au fost identificate și documentate pe parcursul fazei de testare. Din perspectivă econometrică, cea mai notabilă restricție este rigiditatea asumată a modelului în fața matricelor informaționale defectuoase. Atunci când utilizatorul introduce un set de date care prezintă multicolinearitate perfectă sau forțează declanșarea capcanei variabilelor dummy, algoritmul refuză în mod deliberat să calculeze estimatorii. Deși comportamentul funcției *math.lusolve* de a arunca o eroare de execuție în fața unei matrice singulare este o

dovadă a integrității matematice a platformei, utilizatorii fără o pregătire statistică solidă ar putea percepe această oprire de siguranță ca pe un defect software (crash), necesitând pe viitor o interfațare și mai detaliată a erorilor matematice în interfața utilizatorului.

Din punct de vedere tehnic și computațional, utilizarea tipului de date `BigNumber` pentru a asigura o precizie de 64 de zecimale și pentru a eluda limitările standardului IEEE 754 vine cu un compromis de performanță (overhead). Deși insesizabil pe seturi de date educaționale sau de dimensiuni medii (de ordinul miilor de înregistrări), pe măsură ce volumul de observații crește spre scara fenomenelor Big Data (sute de mii de rânduri), operațiile de alocare dinamică a memoriei pentru procesarea textelor matematice și calculul iterațiilor pentru extragerea erorilor standard vor atrage o latență măsurabilă în execuția JavaScript-ului în browser. De asemenea, performanța de scriere asincronă a rapoartelor extinse depinde direct de limitele impuse de coada de execuție a motorului *Excel.run*, care poate fi încetinită de un număr masiv de cereri de formatare cromatică a celulelor în modul educațional detaliat.

5.3. Propuneri și Direcții Viitoare de Dezvoltare

Baza solidă a aplicației oferă multiple oportunități pentru transformarea acestui proiect într-o soluție la standarde profesionale. Principalele direcții de dezvoltare viitoare vizează:

- Pentru a asigura un cod mai curat, decuplat și mai ușor de întreținut, propun rescrierea modulelor și migrarea întregii baze de cod către TypeScript. Tipizarea statică va reduce considerabil riscul de bug-uri. Totodată, este esențială implementarea unor suite de teste automate (unitare și de integrare) pentru a garanta stabilitatea și precizia calculelor matematice la orice actualizare a aplicației.
- În prezent, excepțiile matematice severe (precum coliniaritatea perfectă ce generează o matrice singulară) opresc calculele pentru a preveni rezultatele false. Pe viitor, sistemul poate captura aceste erori și le poate explica utilizatorului prin mesaje clare și prietenoase, indicând exact problema din setul de date. Această îmbunătățire poate fi completată de un mod interactiv de „Training” – un tutor digital care va ghida studenții pas cu pas prin fluxul analizei, explicând deciziile statistice luate de aplicație.
- Raportul econometric poate fi îmbogățit cu funcționalități vizuale. Exploatând mai departe capabilitățile API-ului `Office.js`, aplicația poate genera și poate desena automat grafice direct în foaia Excel (cum ar fi diagrame de dispersie sau histograme

- ale reziduurilor). Astfel, utilizatorii vor avea la dispoziție elemente vizuale gata de a fi integrate în proiecte sau prezentări, direct lângă rezultatele numerice.
- Pentru ca aplicația să fie adoptată la scară largă în mediul universitar, instalarea manuală pe calculatoarele individuale (sideloading) trebuie înlocuită. Trecerea în producție presupune găzduirea componentei web într-o infrastructură cloud sigură (precum Microsoft Azure sau AWS). Odată găzduită public, distribuția (deployment-ul) se va realiza centralizat din portalul Microsoft 365 Admin Center. Departamentul IT va putea astfel să instaleze automat Add-in-ul pe toate conturile instituționale de Excel, oferind profesorilor și studenților acces instantaneu la platformă, fără efort tehnic suplimentar.

CAPITOLUL 6. BIBLIOGRAFIE

- Dr. Chifu, I. C. Reader_Econometrie_24_25, clasa Teams a cursului
- Dr. Simon, L., Dr. Derek , Y., & Dr. Pardoe, I. 5.4 - *A Matrix Formulation of the Multiple Regression Model*. Retrieved from STAT 462:
<https://online.stat.psu.edu/stat462/node/132/>
- Harris, B. *LU Decomposition* . Retrieved from Youtube:
<https://www.youtube.com/@mitocw>
- Lambert, B. *Introduction to the matrix formulation of econometrics*. Retrieved from Youtube: <https://www.youtube.com/@SpartacanUsuals>
- mathjs.org. *BigNumbers*. Preluat de pe mathjs: <https://mathjs.org/docs/index.html>
- Meta Platforms. *Quick Start*. Retrieved from React : <https://react.dev/learn>
- Microsoft. Retrieved from Fluent UI React Components:
<https://storybook.fluentui.dev/react/?path=/docs/concepts-introduction--docs>
- Microsoft. *Office Add-ins documentation*. Retrieved from Microsoft Learn:
<https://learn.microsoft.com/en-us/>
- mozilla.org. *JavaScript data types and data structures*. Retrieved from Resources for Developers, by Developers : <https://developer.mozilla.org/en-US/>
- Wiffin, E. *Floating Point Math*. Retrieved from <https://0.30000000000000004.com/>